



第8讲 数据库索引 (1)



本讲学习什么？

基本内容

1. 为什么需要索引与什么是索引
2. 索引的简单分类
3. B+树索引
4. 散列索引

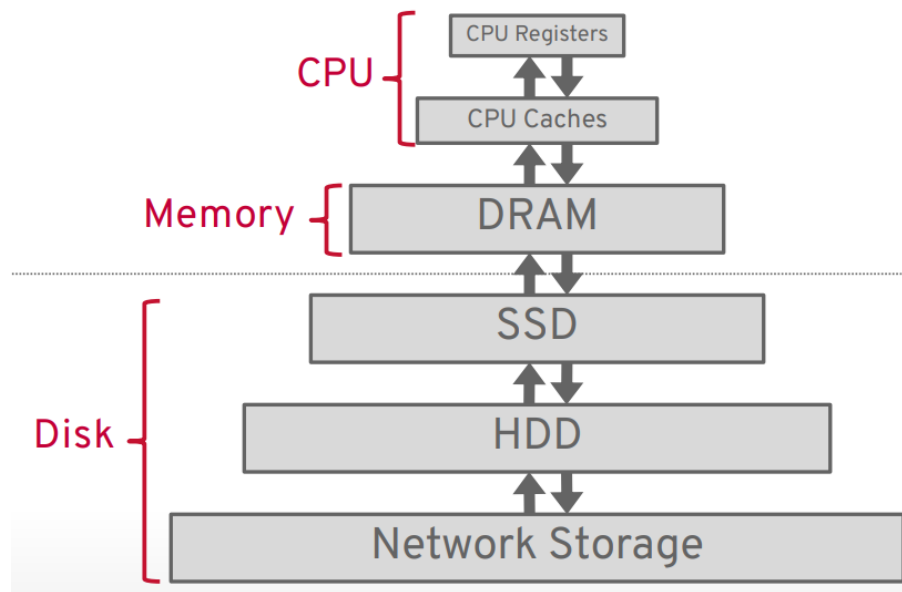
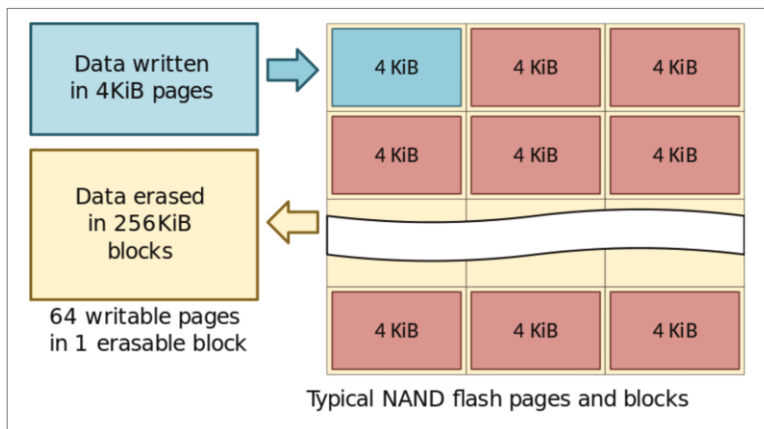
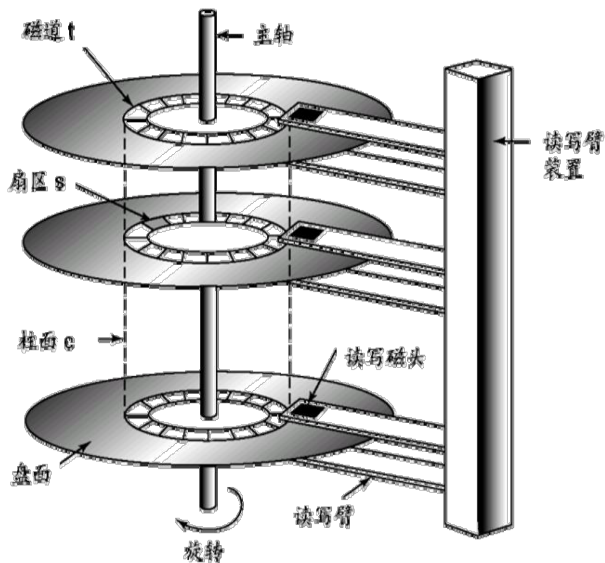
重点与难点

- 理解索引的作用，掌握应用索引改进数据库查询性能的方法
- 理解不同类型索引的概念：稠密索引与稀疏索引，主索引与辅助索引，聚簇索引与非聚簇索引，倒排索引，多级索引等
- 理解B+树索引，怎样建立、维护和利用B+树索引(算法层面)
- 理解散列索引，包括静态散列索引与动态散列索引(算法层面)



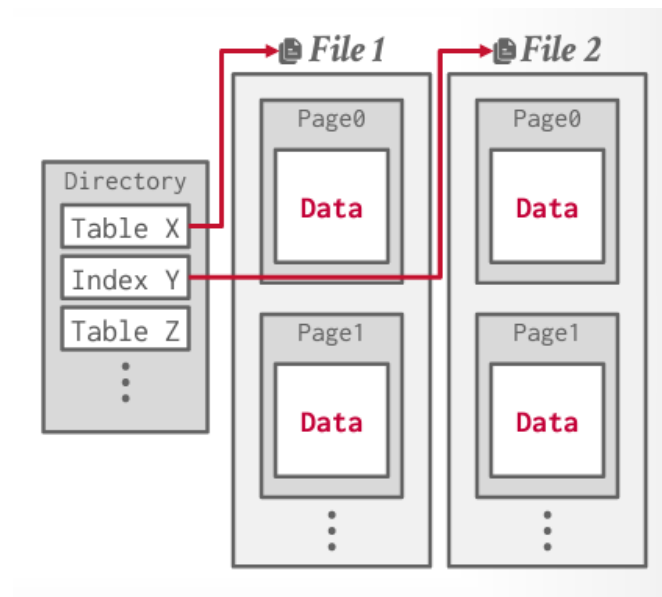
数据库物理存储简介

- 数据库存储大量数据(>>>内存), 需要外部存储: 磁盘、固态硬盘等
- 外存设备读写速度远低于内存, 读写开销大 (I/O Cost)
- 如何高效快速地完成对数据库的查询?
 1. 设计高效的存储结构: 索引(第8讲)和物理存储设计 (第9讲)
 2. 设计高效的查询处理和优化算法 (第10、11讲)



数据库物理存储简介 (后面章节详细讲)

- 数据库里的数据如何存储在外存设备上?
 - 每个表存储在一个或多个数据库文件中
 - 每个文件分为多个页面 (page) 或数据块 (block), 为DBMS基本内存调度单元
 - 每个页面/数据块包含若干条表的记录
- 文件组织方式:
 - 堆文件/无序文件(Heap Files):
 - 排序文件(Sequential Files)
 - 散列文件 (Hash Files)
- 文件的读写和页面调度策略
 - DBMS缓冲区管理
 - 页面调度策略
 - 与操作系统的协同





什么是及为什么需要索引

- 数据库查询:

`SELECT Cname FROM Course WHERE Cno = '103';`

- 如何快速从Course表里找到编号为 = '103' 的记录?
- 方案1: 把Course表的所有记录从文件中读进内存, 逐一扫描
 - 如果Course表存储在很多不同的数据页面中?
 - 如果Course表很大 (例如百万条记录) ?
- 如何降低查询的时间开销? 如何精准找到记录所在存储位置?

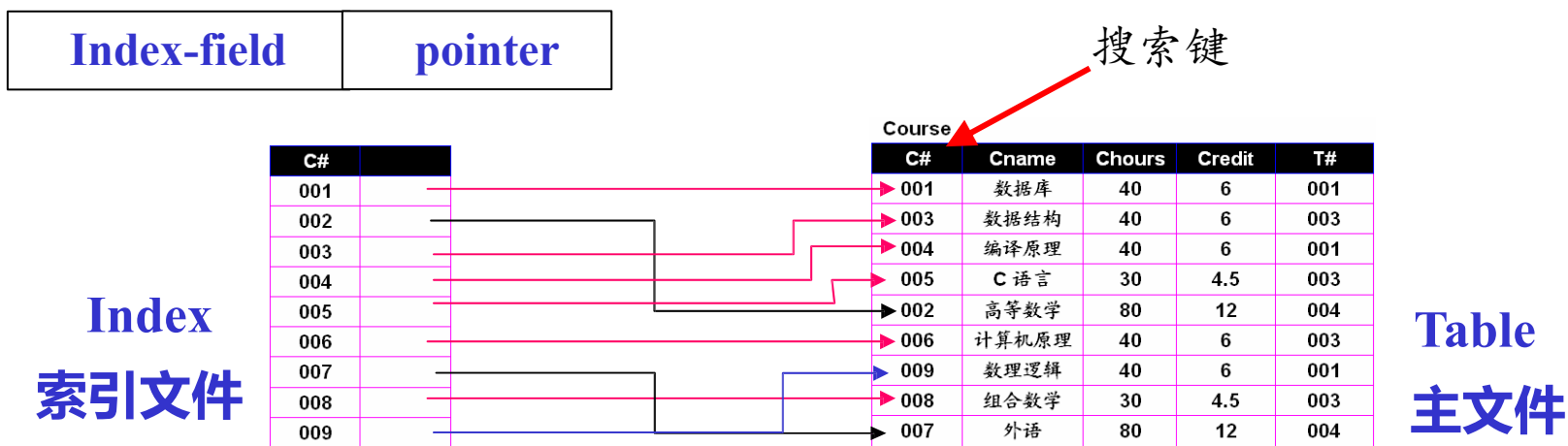


索引概念

索引： 定义在存储表(Table)基础之上，无需检查所有记录，快速定位所需记录的一种**辅助存储结构**，由一系列存储在磁盘上的**索引项**(index entries)组成，每一索引项又由两部分构成：

- **索引字段：** Table中某些列(通常是一列)中的值。该列也称**搜索键**(search key)
- **行指针：** 指向包含索引字段值的表记录在磁盘上的存储位置。

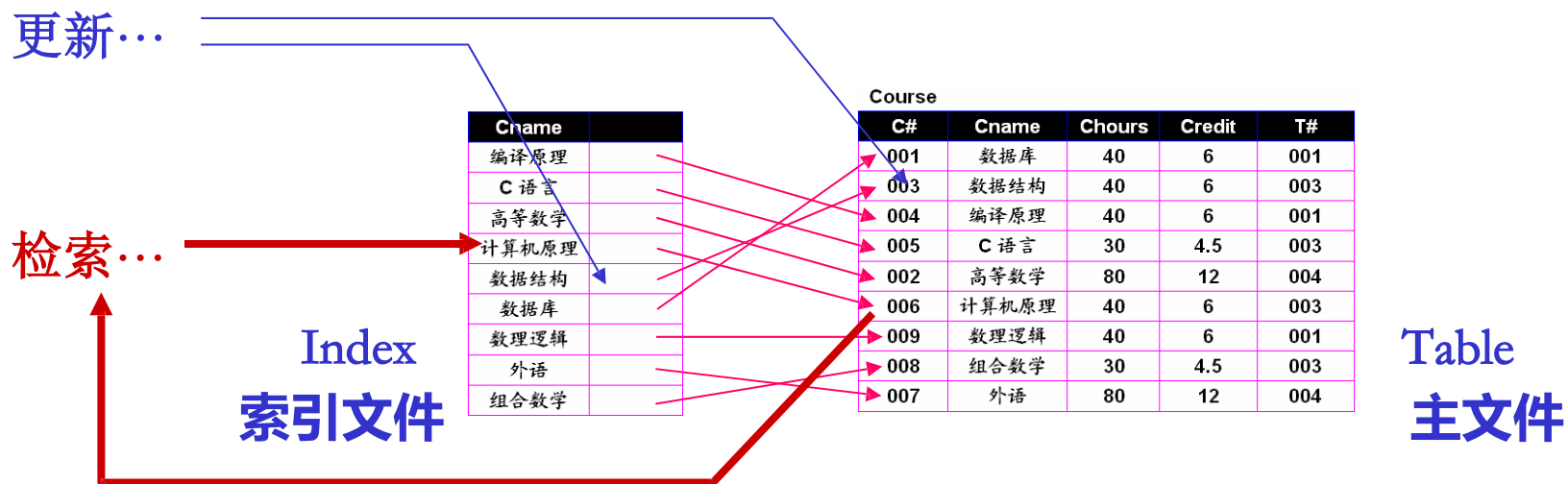
存储索引项的文件为**索引文件**，相对应，存储表又称为**主文件**





索引一般性特点

- 在一个表上，可针对不同属性或属性组合，建立**不同的索引文件**，可建立**多个索引文件**。
(索引字段的值可以是Table中的任何一个属性的值或任何多个属性值的组合值)
- 索引文件比主文件**小很多**。通过检索一个小的索引文件，快速定位后，再有针对性的读取非常大的主文件中的有关记录
- 有索引时，对数据的插入/删除/更新操作**必须同步在**索引文件和主文件进行。否则…





索引应用的评价问题

关于索引应用的评价指标 (evaluation metrics)

- 索引技术应用使检索效率大幅度提高,但同时其也增加了存储空间、使维护负担加重(不仅要维护主文件,而且要维护索引文件)
- 衡量索引性能好坏:

✓ 访问时间开销 (access)

✓ 插入时间开销 (insertion)

✓ 删除时间开销 (deletion)

✓ 空间存储开销 (space overhead)

✓ 支持哪些类型的查询/访问: 等值查询, 范围查询...

对哪些属性
建立索引?

对经常出现在检索条件、
连接条件中的属性可建立索引

WHERE, JOIN ON等

SELECT Tname FROM Teacher WHERE Dno = '03';

SELECT Tname FROM Teacher WHERE Salary >= 1200 AND Salary <=1500;

- 数据库物理设计: 选择哪些属性创建索引, 以及如何创建与维护索引, 如何利用索引改善数据库的运行性能, 是DBA(数据库管理员)的重要职责。



索引分类和比较

□ 排序索引 vs. 散列索引

包括



□ 聚簇索引 vs. 非聚簇索引

等同



等同



□ 主索引 vs. 辅助索引

只能是



一定是



□ 稀疏索引 vs. 稠密索引



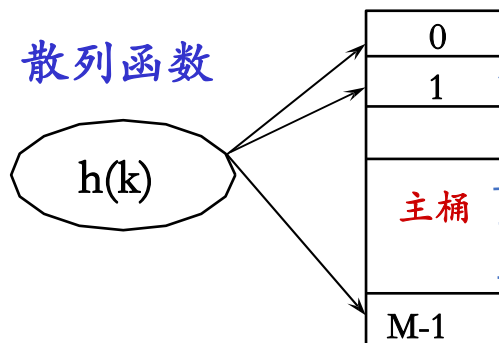
- ## 排序索引文件

Course

C#	Cname	Chours	Credit	T#
▶ 001	数据库	40	6	001
▶ 003	数据结构	40	6	003
▶ 004	编译原理	40	6	001
▶ 005	C 语言	30	4.5	003
▶ 002	高等数学	80	12	004
▶ 006	计算机原理	40	6	003
▶ 009	数理逻辑	40	6	001
▶ 008	组合数学	30	4.5	003
▶ 007	外语	80	12	004

检索值k

散列函数



散列索引文件

Course

C#	Cname	Chours	Credit	T#
001	数据库	40	6	001
003	数据结构	40	6	003
004	编译原理	40	6	001
005	C 语言	30	4.5	003
002	高等数学	80	12	004
006	计算机原理	40	6	003
009	数理逻辑	40	6	001
008	组合数学	30	4.5	003
007	外语	80	12	004

Table 主文件



聚簇索引 vs. 非聚簇索引

- 聚簇索引：主文件按照被索引列进行排序。
- 该列也称排序键/码
- 通常用来索引主键，也可以索引其他列。
- 根据查询需求确定

聚簇索引文件

T#	
001	
003	
004	

聚簇字段值 块指针

排序码 主文件

C#	Cname	Chours	Credit	T#
001	数据库	40	6	001
004	编译原理	40	6	001
009	数理逻辑	40	6	001
008	组合数学	30	4.5	003
003	数据结构	40	6	003
005	C 语言	30	4.5	003
006	计算机原理	40	6	003
002	高等数学	80	12	004
007	外语	80	12	004

索引值与表中搜索键排序相同

非聚簇索引文件

T#是“非主码字段”有重复值排序存储

T#	
001	
001	
001	
003	
003	
003	
003	
004	
004	

索引字段值 记录指针

主文件

C#	Cname	Chours	Credit	T#
001	数据库	40	6	001
002	高等数学	80	12	004
003	数据结构	40	6	003
004	编译原理	40	6	001
005	C 语言	30	4.5	003
006	计算机原理	40	6	003
007	外语	80	12	004
008	组合数学	30	4.5	003
009	数理逻辑	40	6	001

主文件没有基于“非主码的字段”T#排序存储

- 非聚簇索引：主文件未按照被索引列排序。
- 通常用来索引非主键。



主索引 vs. 辅助索引

- 主索引(Primary Index): 对**排序文件**的排序码/键建立的索引。一个表只有一个主索引(因为物理上, 每个表只能按照一种方式排序)。**主索引等同于聚簇索引。**
- 通常建立在排序主文件的主键上 (主键的索引和主索引并不是同一概念, 二者可能不同)。

主索引 通常是对每一存储块/页有一个索引项, 索引项的总数和存储表所占的存储块数目相同, 存储表的每一存储块的第一条记录, 又称为锚记录(anchor record), 或简称为块锚(block anchor)

- 主索引的索引字段值为块锚的索引字段值, 而指针指向其所在的存储块。

主索引文件

C#	
001	
004	
007	

块锚主码值 块指针

C#	Cname	Chours	Credit	T#
001	数据库	40	6	001
002	高等数学	80	12	004
003	数据结构	40	6	003
004	编译原理	40	6	001
005	C 语言	30	4.5	003
006	计算机原理	40	6	003
007	外语	80	12	004
008	组合数学	30	4.5	003
009	数理逻辑	40	6	001

主文件



辅助索引 (Secondary Index)

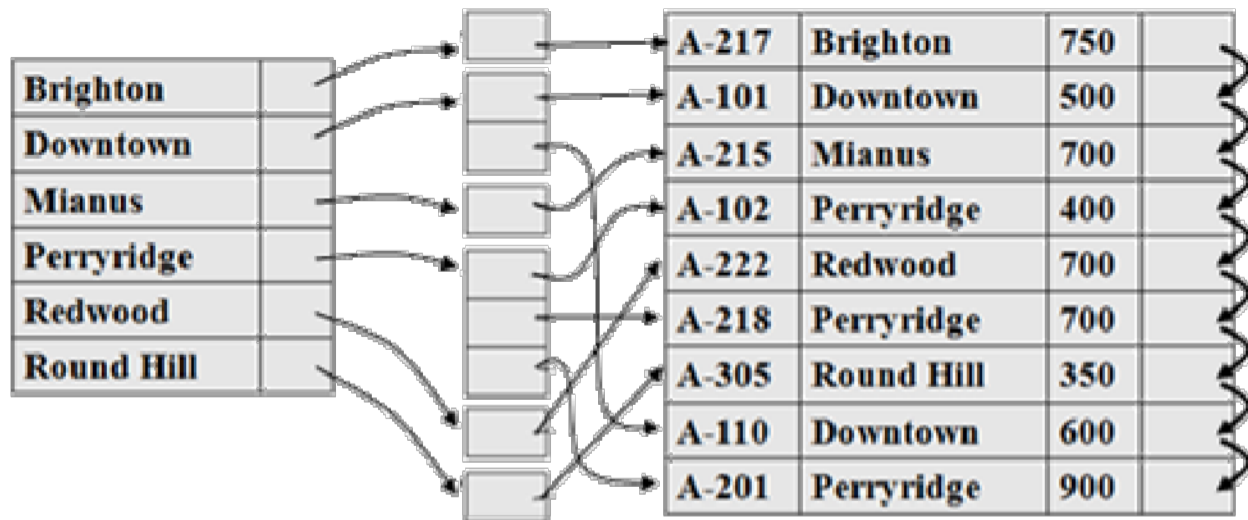
辅助索引 **等同于非聚簇索引**。是定义在主文件的任一或多个**非排序字段上**的辅助存储结构。

➤ 辅助索引通常是对某一非排序字段上的每一个不同值有一个索引项：

每个索引项都对应指向包含该记录的块或该记录本身；

➤ 当非排序字段为索引字段时，如该字段值**不唯一**，则要采用一个**类似链表**的结构来保存包含该字段值的所有记录的位置。

辅助索引文件

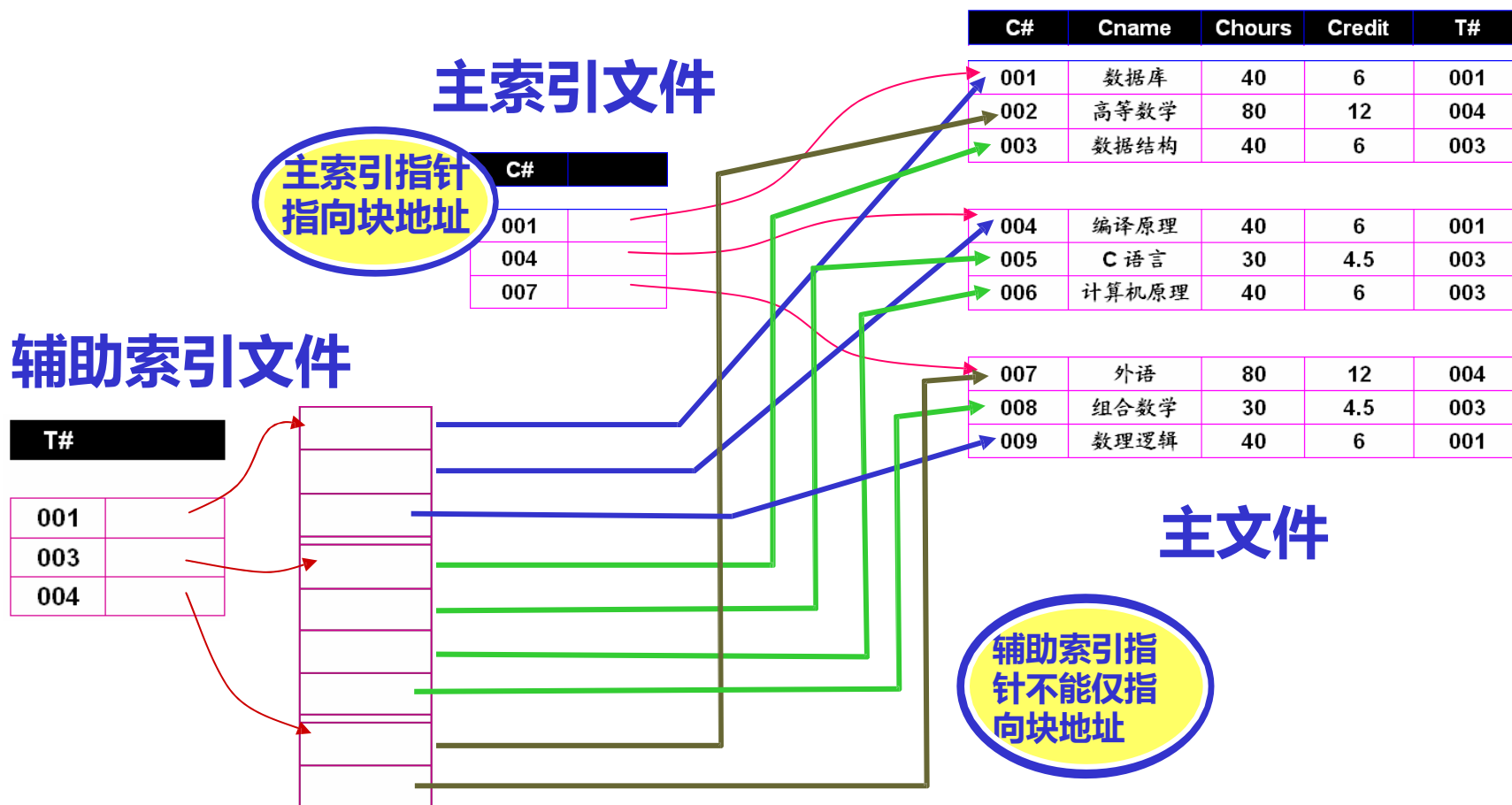


主文件



主索引 vs. 辅助索引

- 一个主文件仅可以有一个主索引，但可以有多个辅助索引
- 主索引通常建立于主码/排序码上面；辅助索引建立于其他属性上面
- 可以利用主索引重新组织主文件数据，但辅助索引不能改变主文件数据





稠密索引与稀疏索引

- 对于主文件中搜索键的每个值，都有一个索引项和它对应。这样的索引称**稠密索引(dense index)**
- 对于主文件中搜索键的部分值，有索引项和它对应，这样的索引称**非稠密索引(undense index)**或**稀疏索引(sparse index)**。通常稀疏索引指针指向每个数据块地址。
- 主索引一般是**稀疏索引**，稀疏索引**只能是**主索引。辅助索引则必须是**稠密索引**。
- 稀疏索引**空间占用小**，但**查询速度更慢**。稠密索引**空间占用大**，但**查询效率更高**。

稠密索引

Brighton	→	A-217	Brighton	750	
Downtown	→	A-101	Downtown	500	
Downtown	→	A-215	Mianus	700	
Mianus	→	A-102	Perryridge	400	
Perryridge	→	A-222	Redwood	700	
Perryridge	→	A-218	Perryridge	700	
Perryridge	→	A-305	Round Hill	350	
Redwood	→	A-110	Downtown	600	
Round Hill	→	A-201	Perryridge	900	

索引文件中包含了主文件对应字段的**所有**不同值

稀疏索引

Brighton	→	A-217	Brighton	750	
Mianus	→	A-101	Downtown	500	
Redwood	→	A-110	Downtown	600	
	→	A-215	Mianus	700	
	→	A-102	Perryridge	400	
	→	A-201	Perryridge	900	
	→	A-218	Perryridge	700	
	→	A-222	Redwood	700	
	→	A-305	Round Hill	350	

索引文件中包含了主文件对应字段的**部分**不同值



稀疏索引定位记录

稀疏索引如何定位记录

- 定位索引字段值为 K 的记录，需要
 - ✓ 首先找相邻的，小于 K 的，最大索引字段值，所对应的索引项
 - ✓ 读取该记录对应的数据块，顺序进行扫描
- 稀疏索引的使用要求——主文件必须是按对应索引字段属性**排序存储，即主索引。**

稀疏索引

Brighton	
Mianus	
Redwood	

索引文件中不存在搜索码的值，不代表主文件中没有对应搜索码的记录

A-217	Brighton	750	
A-101	Downtown	500	
A-110	Downtown	600	
A-215	Mianus	700	
A-102	Perryridge	400	
A-201	Perryridge	900	
A-218	Perryridge	700	
A-222	Redwood	700	
A-305	Round Hill	350	



索引分类和比较

□ 排序索引 vs. 散列索引

包括



□ 聚簇索引 vs. 非聚簇索引

等同



□ 主索引 vs. 辅助索引

等同



只能是



□ 稀疏索引 vs. 稠密索引

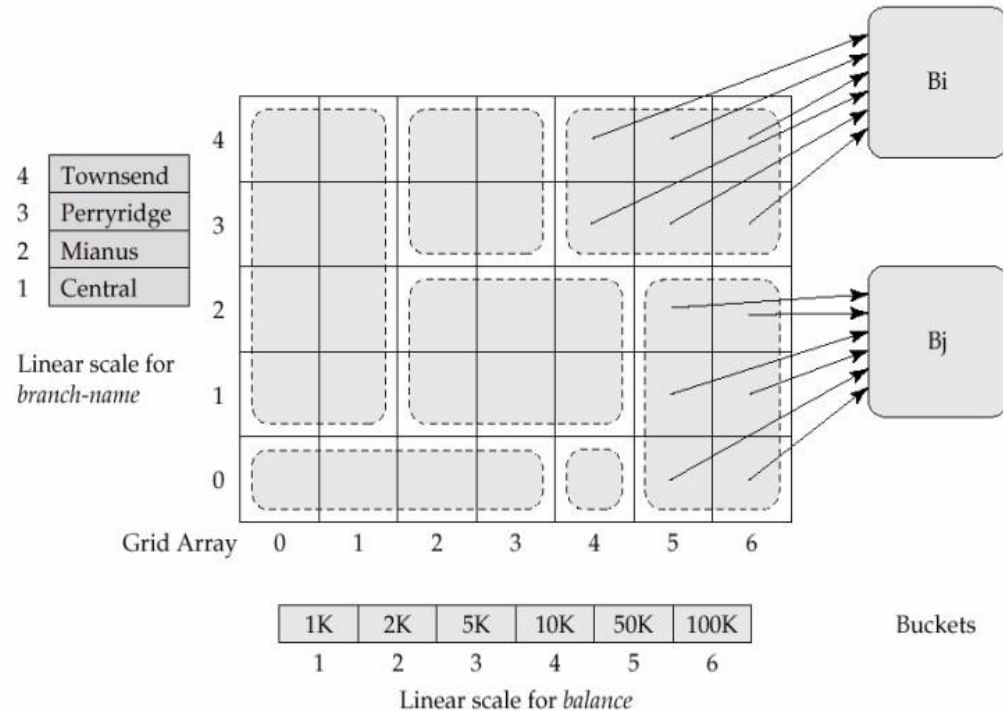
一定是





更多索引结构和类别

- 多级索引：当索引项比较多时，**可以对索引再建立索引**，依此类推，形成多级索引
 - ✓ 常见的多级索引形式，如**B树/B+树索引**，以树型数据结构来组织索引项等
- 多属性索引：索引字段由Table的多个属性值组合在一起形成的索引
- **散列索引**：使用散列技术组织索引
- 网格索引(Grid file)：使用多索引字段进行交叉联合定位与检索
- 位图索引(Bitmap Index)：对枚举类型或离散变量进行索引





SQL语言中的索引创建与维护

SQL语言关于索引的基本知识

- 当定义Table后，如果定义了主键(Primary Key)，则系统将自动为其创建主索引；
- 索引也可以由用户创建、撤消。DBMS将自动维护更新所有的索引，保持与Table中数据的一致性。
- Table被删除后其上定义的索引也同时被删除。
- MySQL中创建索引的命令示例：

CREATE INDEX idxSname **ON** student(sname);

--在student表中创建一个基于Sname的索引

CREATE INDEX idxSnamcl **ON** student(sage) **USING BTREE**;

--在student表中创建一个基于sage的B树索引

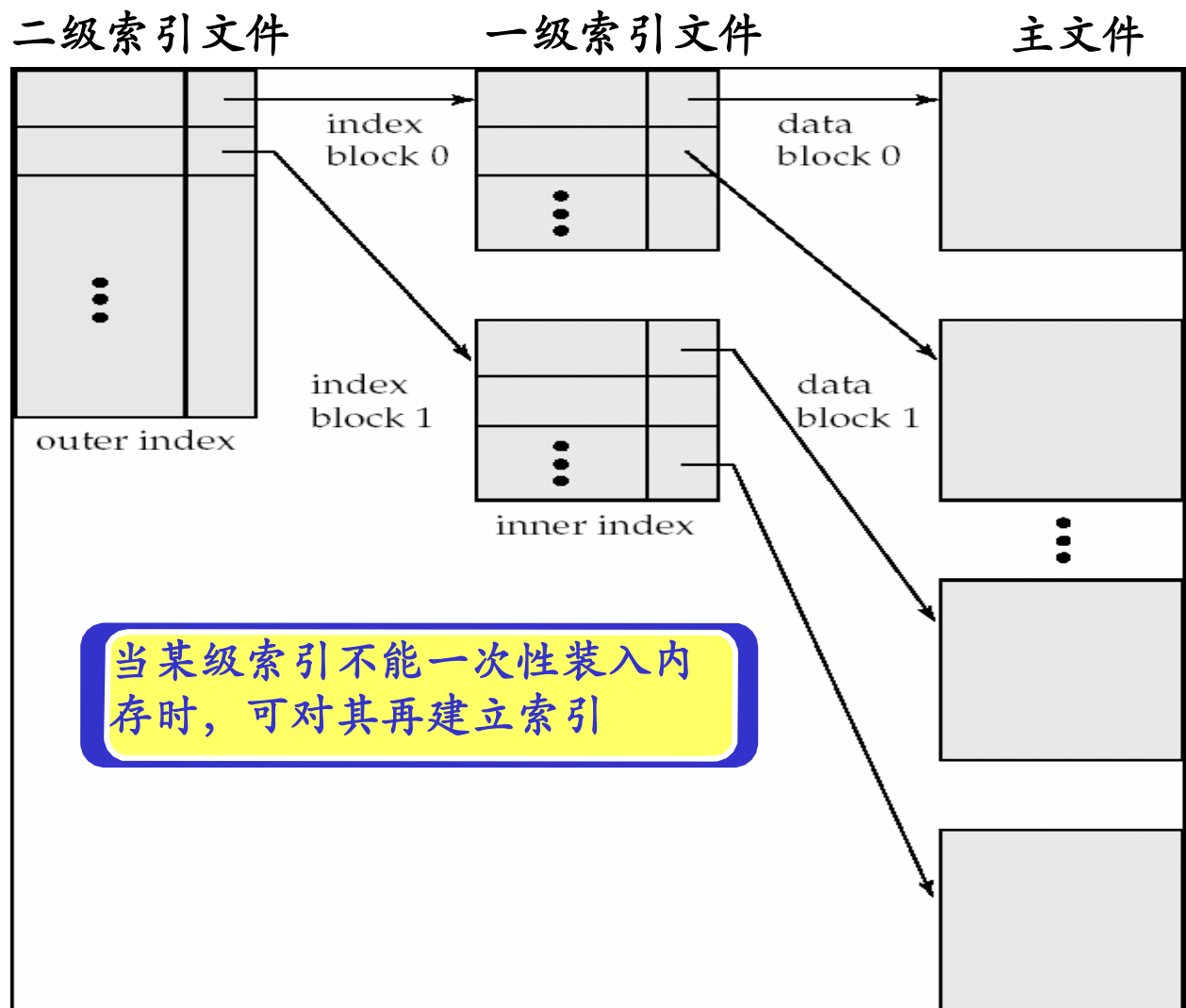
DROP INDEX indexname;

--删除名为indexname的索引



B+树(B+ Tree)

多级索引：当索引项比较多时，可以对索引再建立索引，依此类推，形成多级索引。



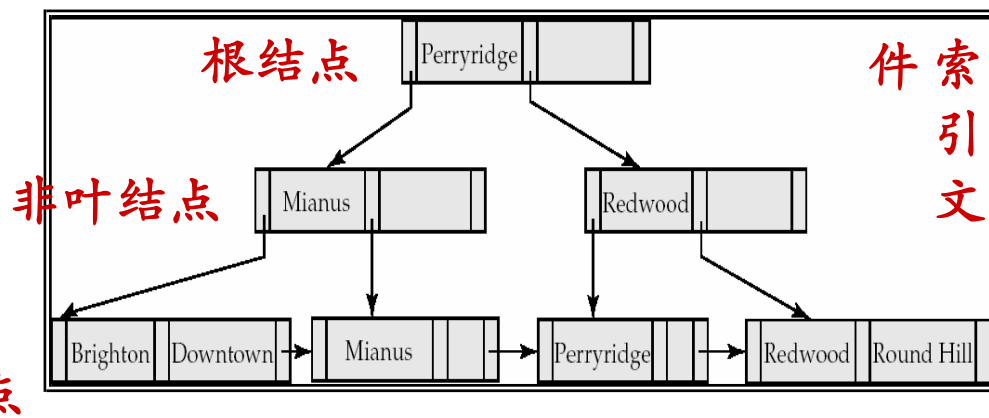


B+树索引：一种以树型数据结构来组织索引项的多级索引

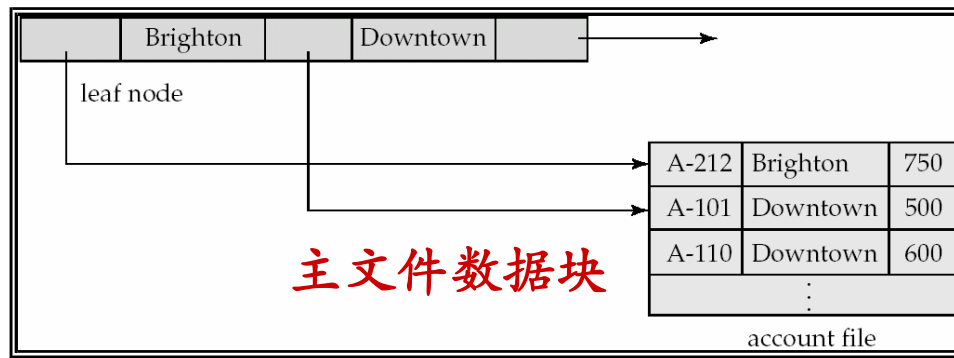


每个树节点中索引项的结构

- K_i —索引字段值
- P_j —指针，指向索引块 或 数据块 或 数据块中记录的指针



索引文件的叶子结点



- 能够自动保持与主文件大小相适应的树的层次
- 每个索引块的指针利用率都在 50%-100%之间



B+树的存储约定

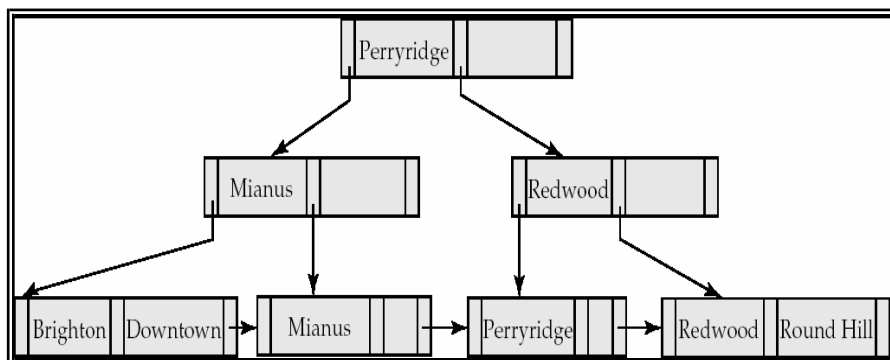
一个树节点存放多少个索引项(n 的大小, 即指针的个数)?



有 $n-1$ 个索引项(<索引字段值 K_i , 指针 P_i >)+1个指针(P_n);

索引字段值 x 在 $K_{i-1} \leq x < K_i$ 的
由 P_i 指向 (左) ; 而 $K_i \leq x < K_{i+1}$
的由 P_{i+1} 指向 (右) 。

每个树节点存储大小 = 1个DBMS存储块
每次按整个节点进行读/写操作



示例 存储块 = 4096 Byte
整数型索引字段值 = 4 Byte
指针 = 8 Byte
则: n 应满足 $4(n-1) + 8n \leq 4096$
 n 取最大值
 $n=341$



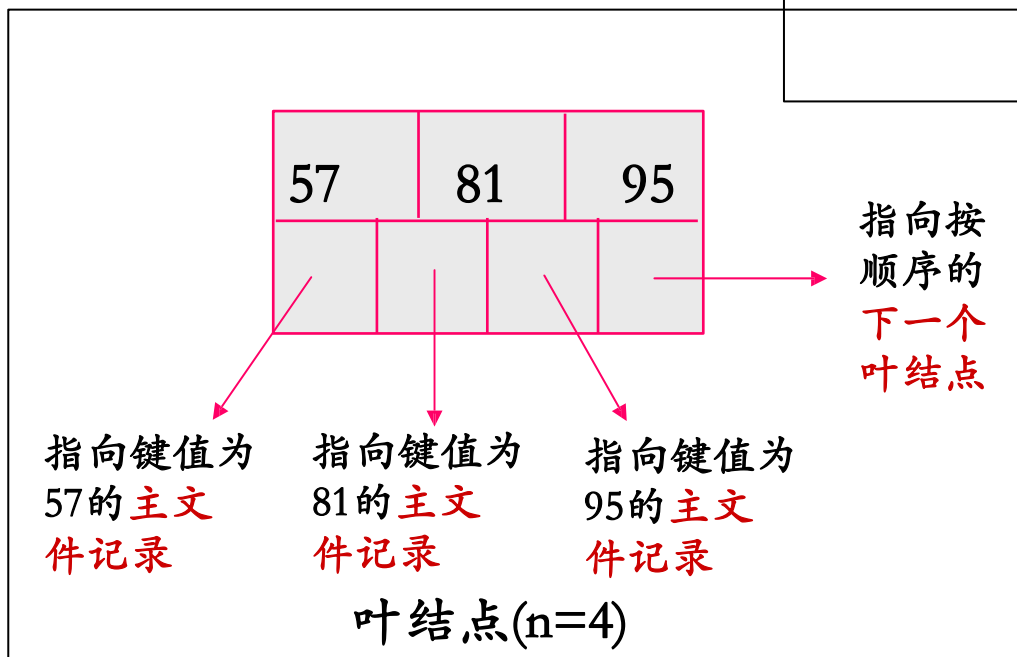
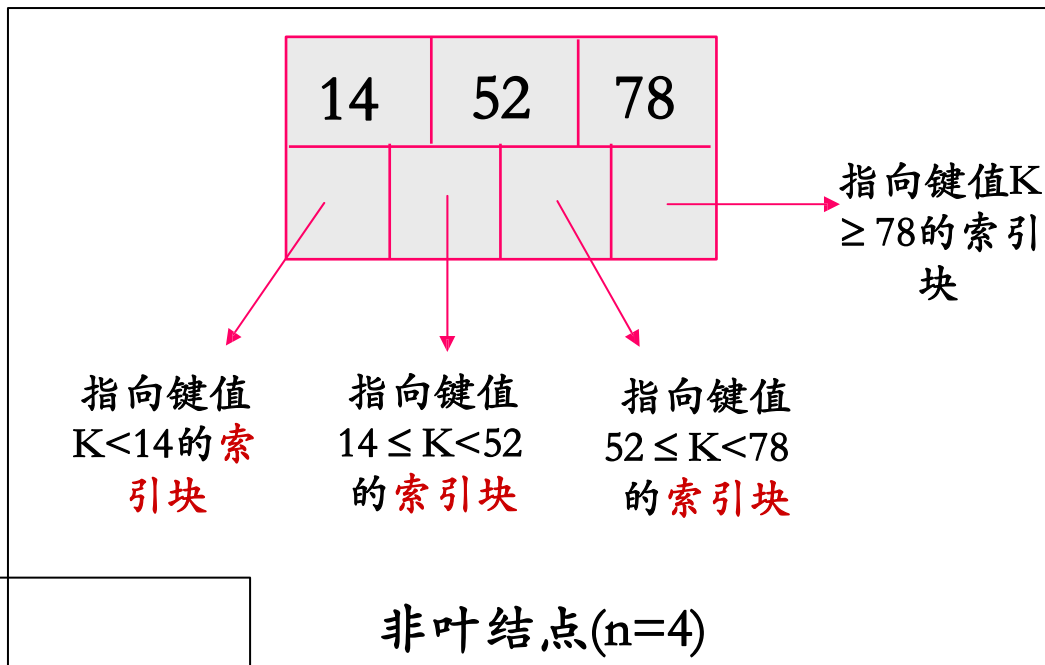
B+树的存储约定

<https://www.cs.usfca.edu/~galles/visualization/BPlusTree.html>

叶结点和非叶节点的指针指向什么？

- 非叶结点指针指向索引块，叶结点指针指向主文件的数据块或数据记录

叶结点的最后一个指针始终指向其下一个数据块



- 一索引块实际使用的索引指针个数 d ，满足(根结点除外):

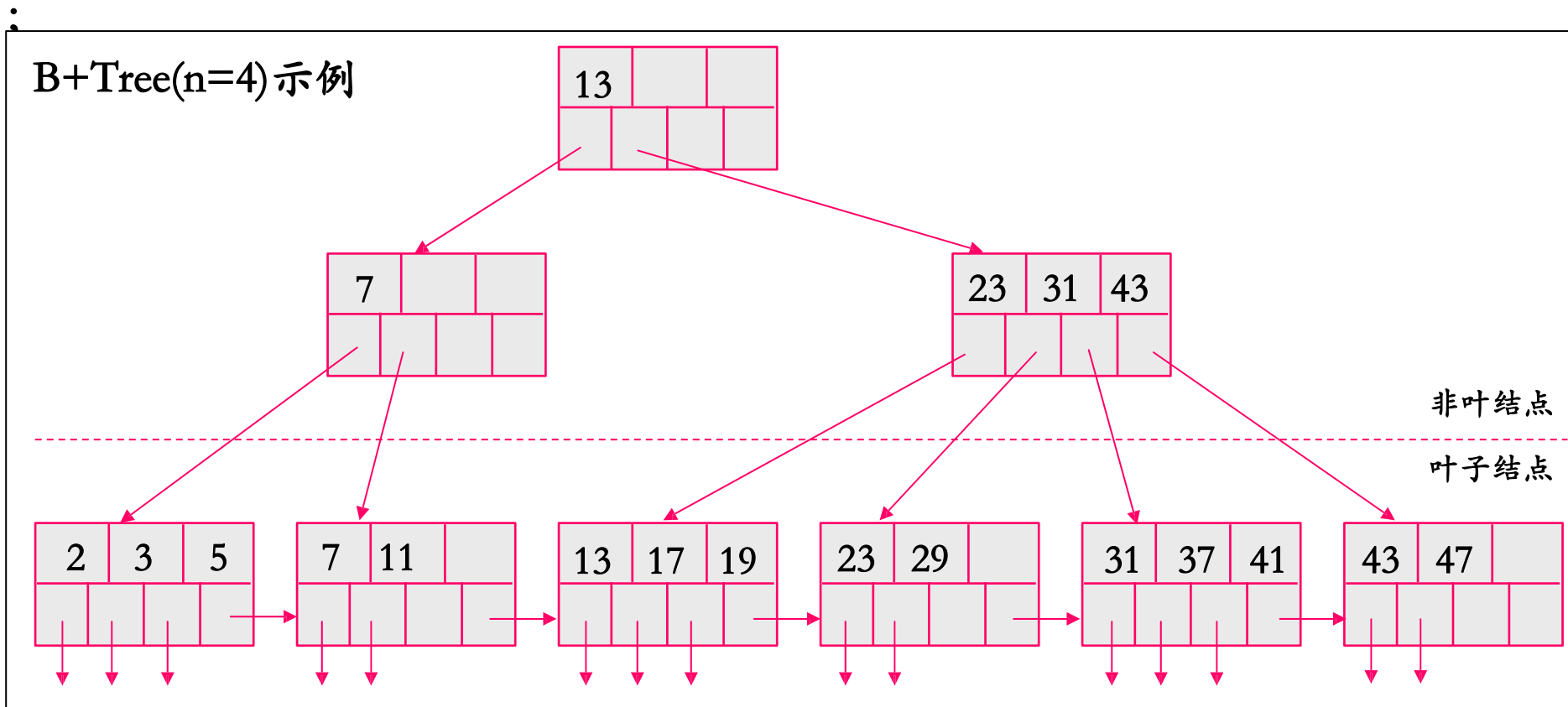
$$n/2 \leq d \leq n。$$

- 根结点至少2个指针被使用。



B+树的存储约定

- 索引字段值重复出现于叶结点和非叶结点 (如下例中23)



- 指向主文件的指针**仅出现**于叶结点;
- 所有叶结点即可**覆盖所有**键值的索引;
- 索引字段值在叶结点中是按顺序排列的;

仅叶结点块的集合就是主文件完整的索引, 即: 最低一级(或一层)索引



B+树的存储约定

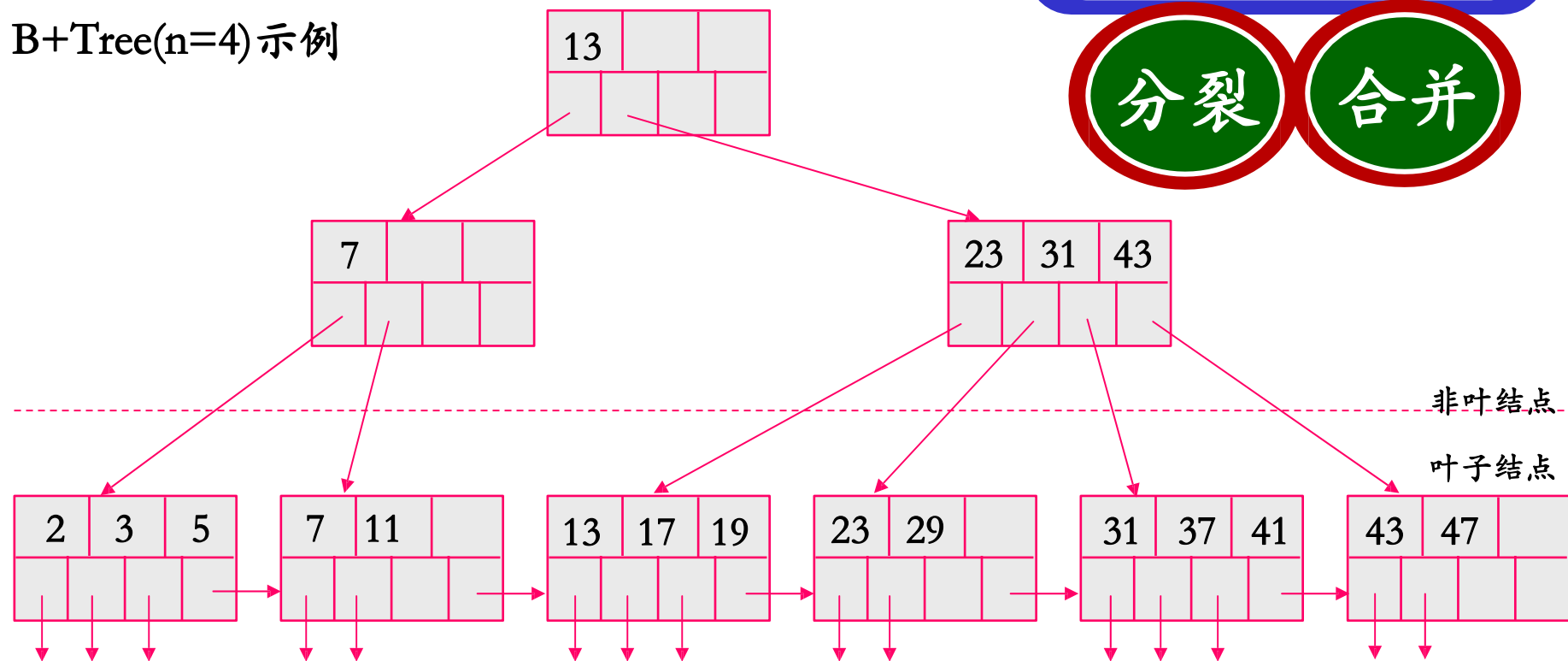
- 级数(或层数)相同--平衡。如何保证?
- 插入/删除记录时, 伴随着结点的分裂与合并;
- 分裂与合并将调整部分结点块中的索引项
(需有算法支持)

- 能够自动保持与主文件大小相适应的树的层次
- 每个索引块的指针利用率都在50%-100%之间

分裂

合并

B+Tree(n=4) 示例

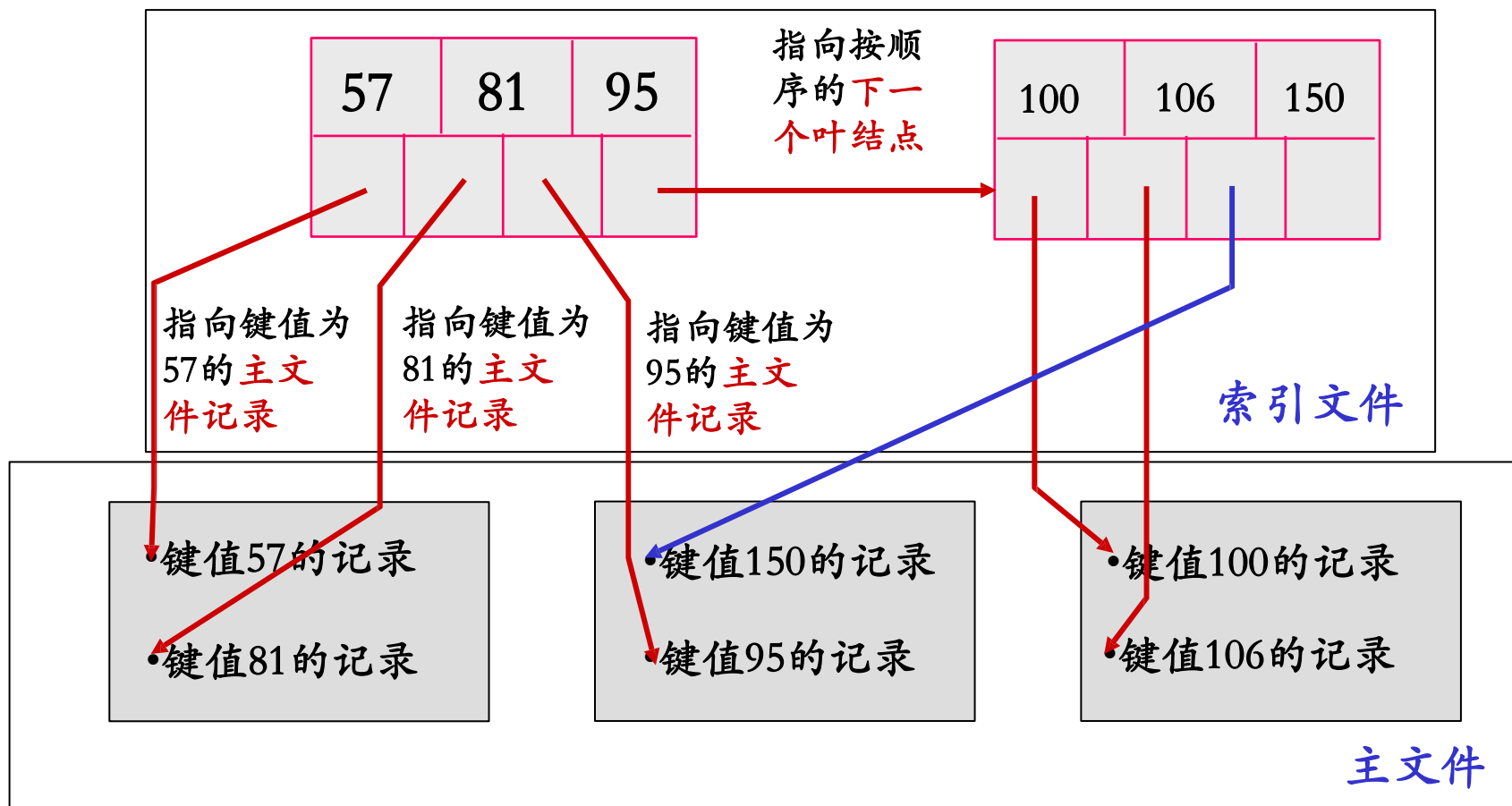




用B+树建立不同的索引

I. 用B+树建立键属性稠密索引

- 索引字段是主文件的主键，索引是稠密的。主文件可以按主键排序，也可以不按主键排序。指针指向的是记录。

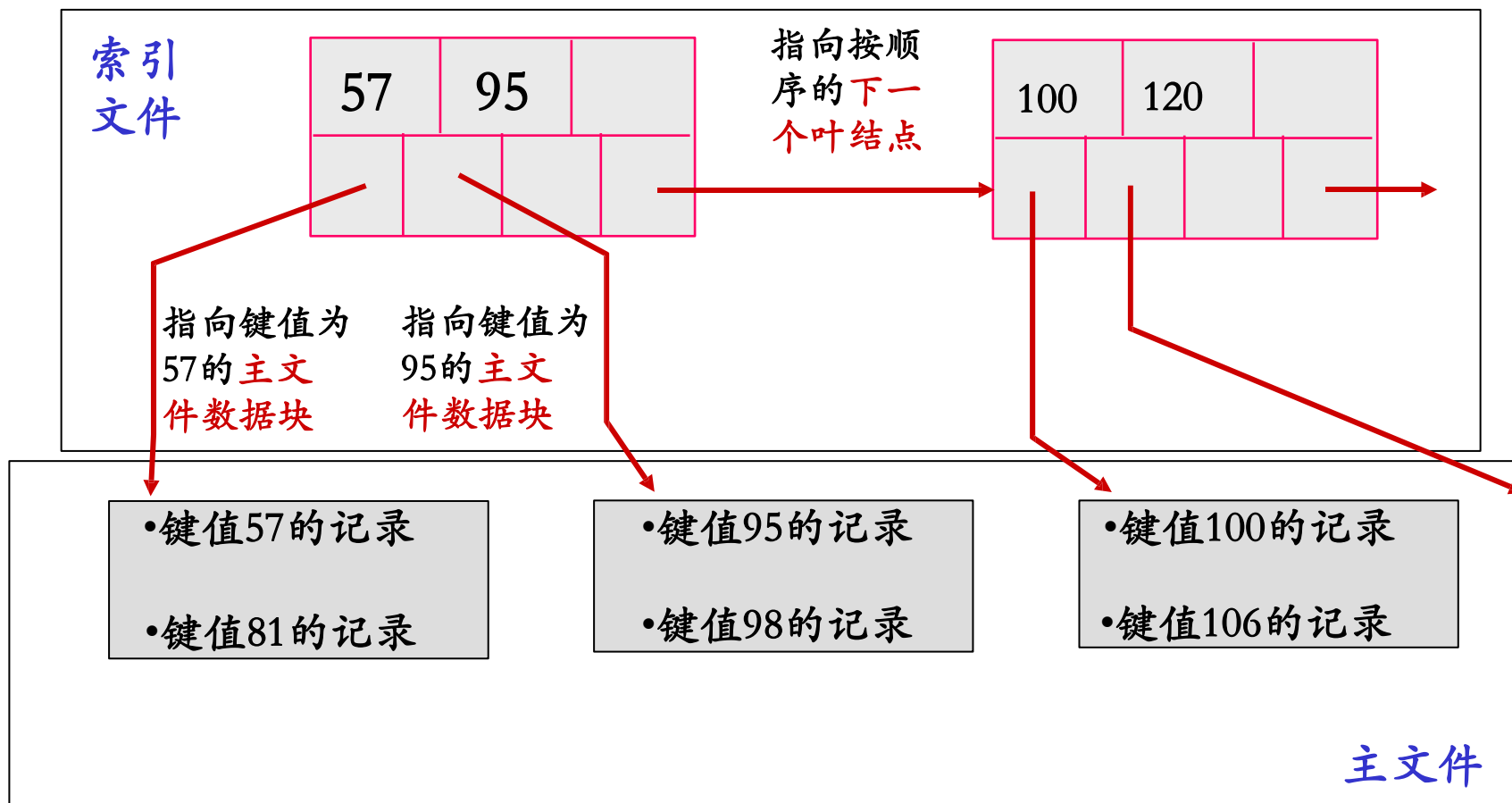




用B+树建立不同的索引

II. 用B+树建立稀疏索引(或主索引)

- 索引字段是主文件的主键，索引是稀疏的。主文件必须按主键排序。
- 指针指向的是数据块





- 索引字段是主文件的非键属性。主文件**不按此非键属性排序**
- 索引文件的索引字段值是有重复的。指针指向的是记录。





B+树说明

□ 索引键存在重复值时的处理方法：

- 所有重复值存在同一个叶节点内，使用溢出块来装载额外值。
- 或者将重复值全部插入B+树内。
 - 此时重复值有可能出现在多个节点中
 - 可能不满足指针的取值范围要求
 - 修改搜索算法，左右子树都搜索

□ 同样的数据可能有不同的树结构

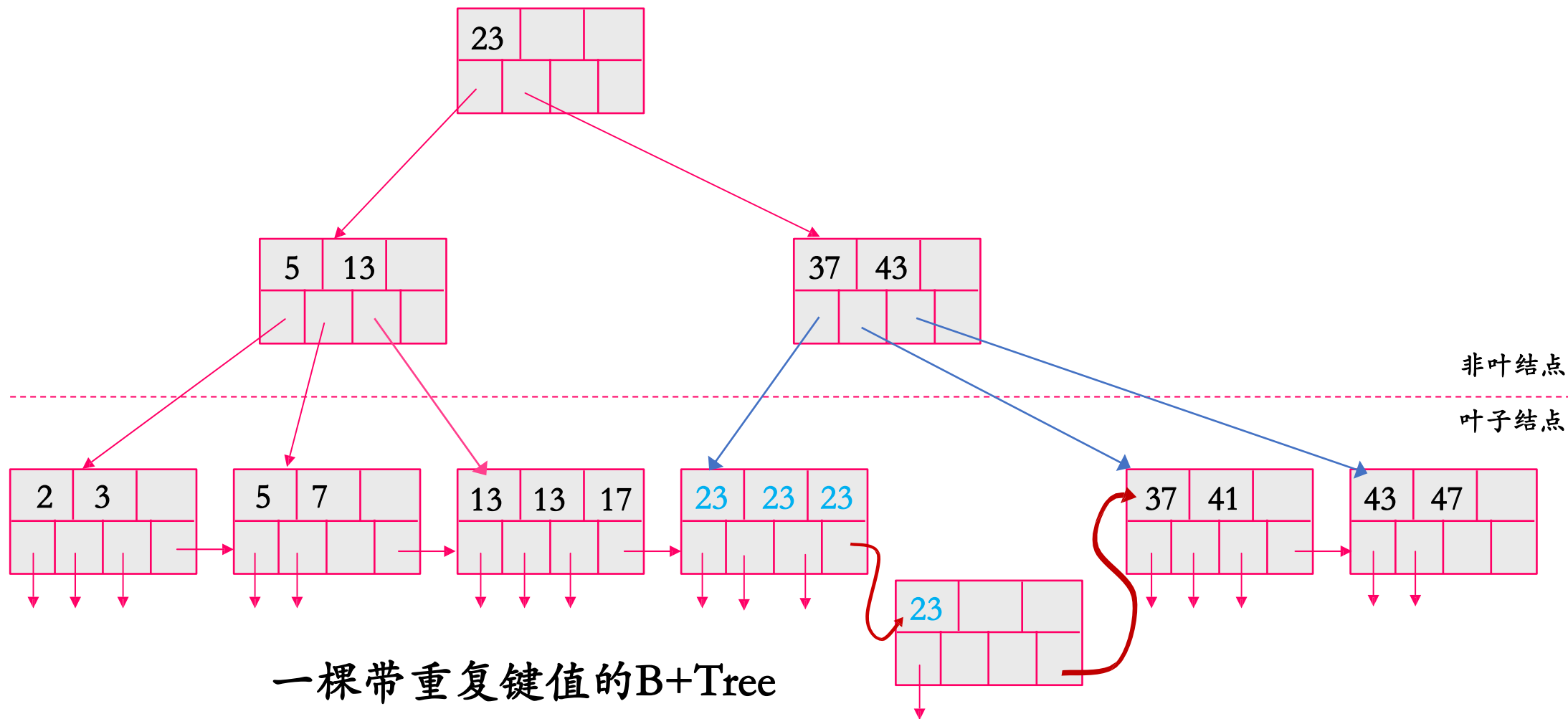
- 插入顺序不同，树的结构可能不同



用B+树建立不同的索引

IV. 用B+树建立非键属性稠密索引

- 索引字段带有重复值（如下图中的23）的B+树的非叶结点示例（使用溢出块）

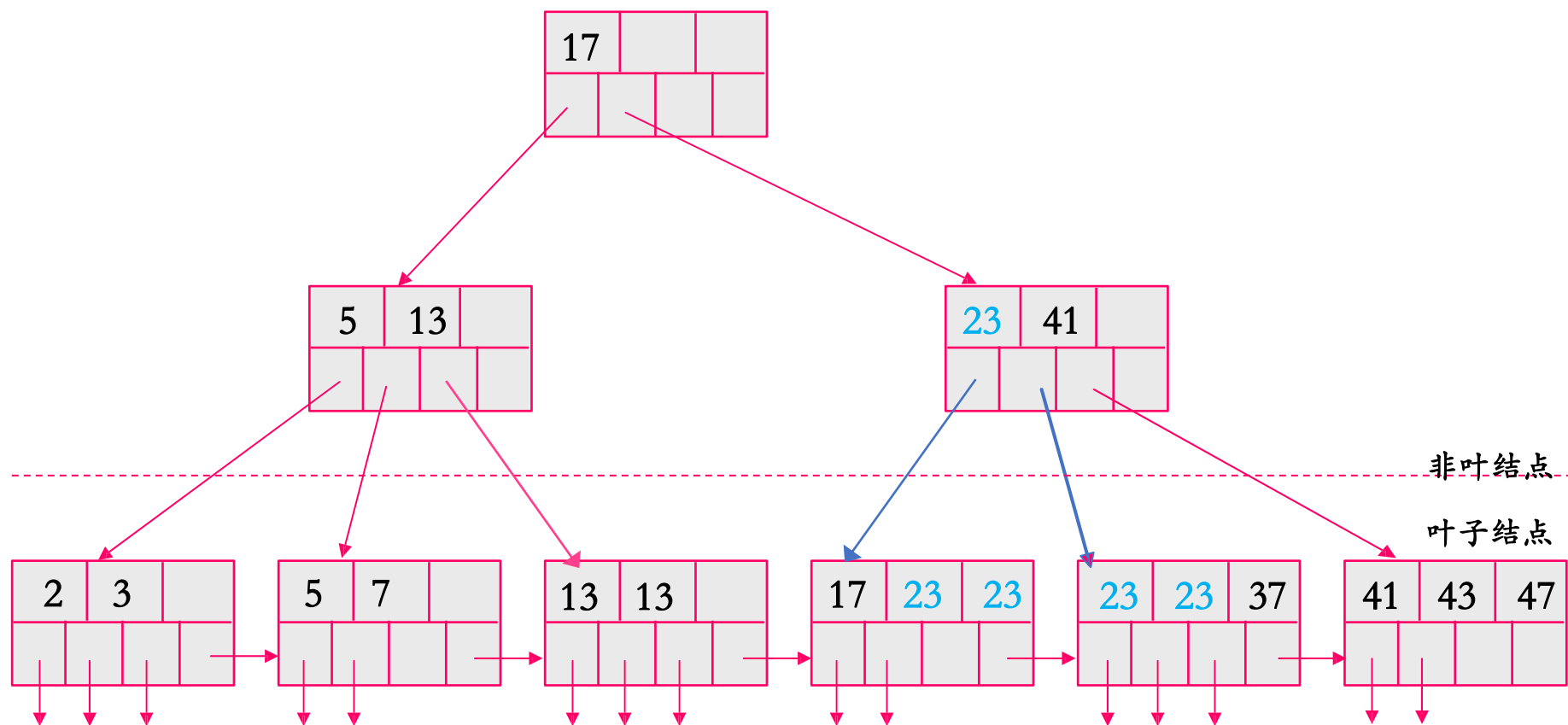




用B+树建立不同的索引

IV. 用B+树建立非键属性稠密索引

- 索引字段带有重复值（如下图中的23）的B+树的非叶结点示例（重复值全部插入索引）



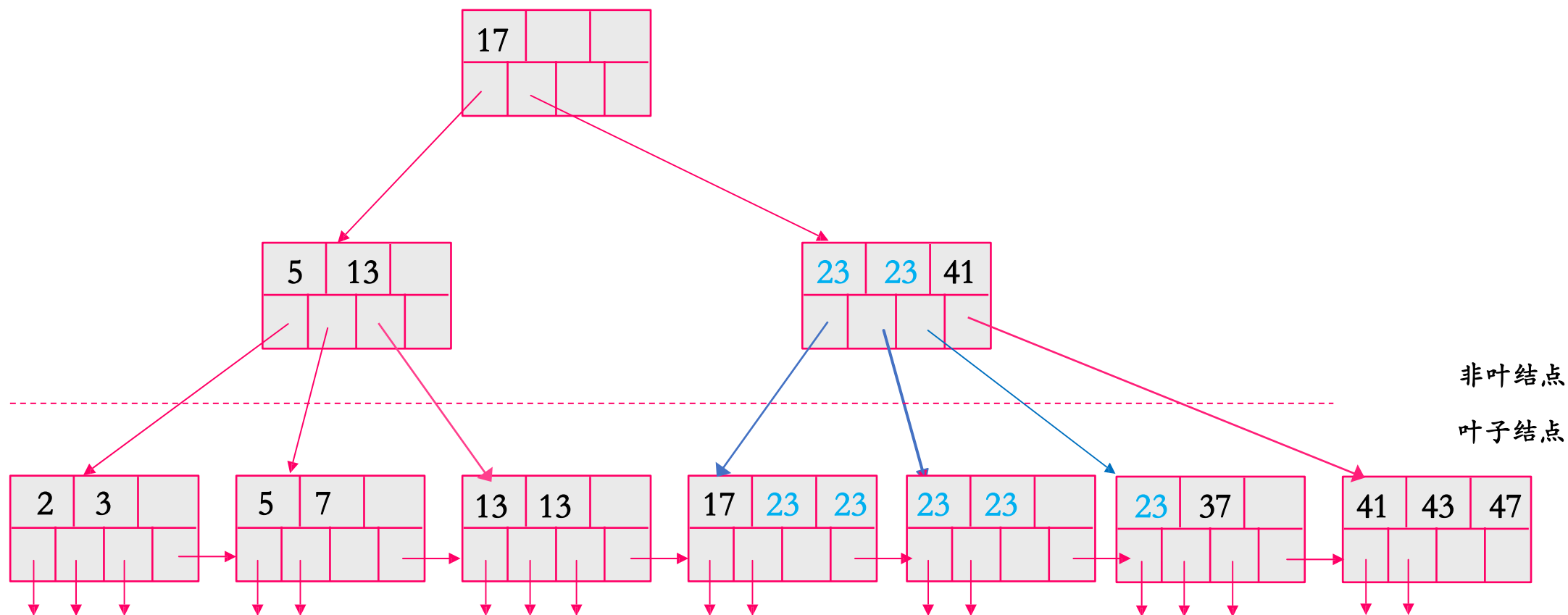
一棵带重复键值的B+Tree



用B+树建立不同的索引

IV. 用B+树建立非键属性稠密索引

- 索引字段带有重复值（如下图中的23）的B+树的非叶结点示例（重复值全部插入索引）



一棵带重复键值的B+Tree



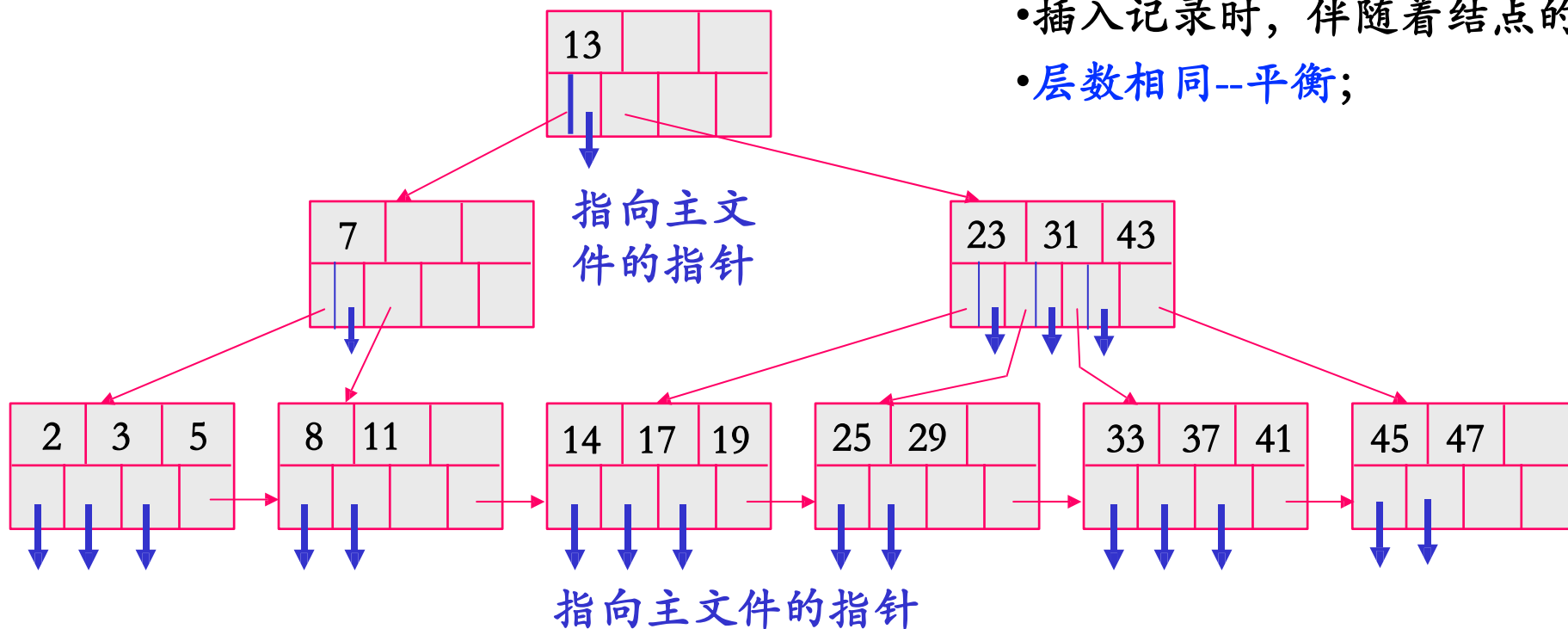
B+树 vs. B树

B Tree

- 索引字段值**仅出现一次**或者在叶结点或者在非叶结点;
- 指向主文件的指针出现于**叶结点或非叶结点**;
- 所有结点**才能覆盖所有键值的索引。

相同点

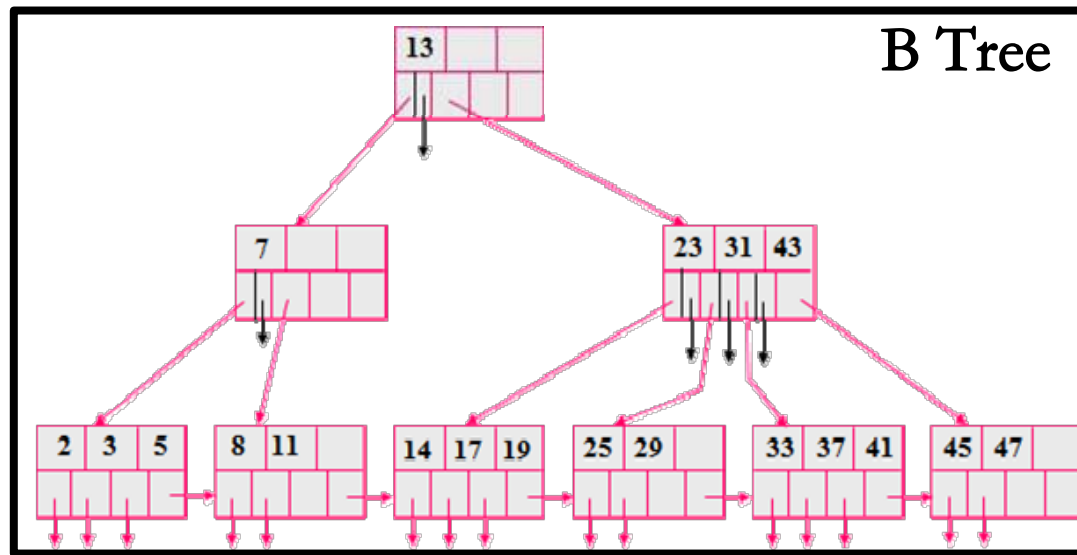
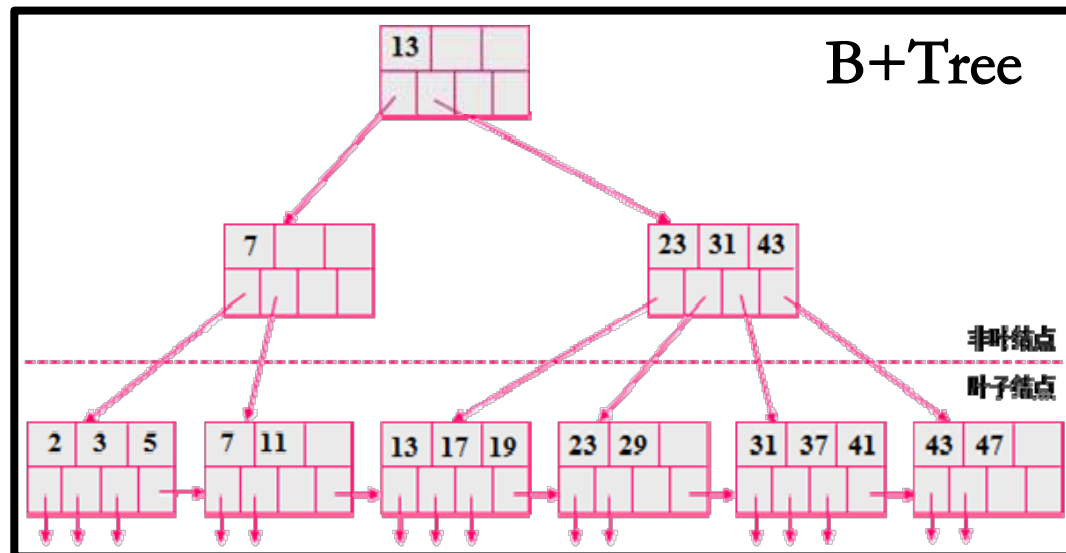
- 插入记录时, 伴随着结点的**分裂与合并**;
- 层数相同--平衡**;





B+树 vs. B树

- 一块中存放的索引项个数是否相同?
- 索引字段值都出现在哪里?
- 指向主文件的指针存在哪里?
- 分裂与合并的方法是否一致?





B+树还要考虑什么

利用B+树检索的算法

```
function find( value V)
/* 假设没有重复码返回叶结点 C 和索引 i 使得 C. Pi 指向第一条搜索码值等于 V 的记录 */
置 C = 根结点
while C 不是叶结点 begin
    令 i = 满足  $V \leq C. K_i$  的最小值
    if 没有这样的 i then begin
        令 Pm = 结点中最后一个非空指针
        置 C = C. Pm
    end
    else if ( V = C. Ki )
        then 置 C = C. Pi+1
    else C = C. Pi / *  $V < C. K_i$  */
end
/* C 是叶结点 */
设 i 是满足  $K_i = V$  的最小值
if 有这样的 i 存在
    then 返回( C, i )
else 返回空 /*

procedure printAll( value V)
/* 打印所有搜索码值等于 V 的记录 */
置 done = false
置( L, i ) = findFirst( V )
if( ( L, i ) 为空 ) return
repeat
    repeat
        打印指针 L. Pi 指向的记录
        置 i = i + 1
    until ( i > L 中码的数目 或 L. Ki > V )
    if ( i > L 中码的数目 )
        then L = L. Pn
        else 置 done = true;
until ( done or L 为空 )
```

图 11-11 B⁺树上的查询

利用B+树删除记录的算法

```
procedure delete( value K, pointer P)
找到包含( K, P )的叶结点 L
delete_entry( L, K, P)

procedure delete_entry( node N, value K, pointer P)
从 N 中删除( K, P )
if ( N 是根结点 and N 只剩下一个子结点 )
    then 使 N 的子结点成为该树的新的根结点并删除 N
else if ( N 中值/指针太少 ) then begin
    令 N' 为 parent( N ) 的前一子女或后一子女
    令 K' 为 parent( N ) 中指针 N 和 N' 之间的值
    if ( N 和 N' 中的项能放在一个结点中 )
        then begin /* 合并结点 */
            if ( N 是 N' 的前一个结点 ) then swap_variables( N, N' )
            if ( N 不是叶结点 )
                then 将 K' 以及 N 中所有指针和值附加到 N' 中
            else 将 N 中所有 ( Ki, Pi ) 对附加到 N' 中; 令 N'. Pn = N. Pn
                ( parent( N ), K', N ); 删除结点 N
        end
    else begin /* 重新分布: 从 N' 借一个索引项 */
        if ( N' 是 N 的前一个结点 ) then begin
            if ( N 是非叶结点 ) then begin
                令 m 满足: N'. Pm 是 N' 的最后一个指针
                从 N' 中去除( N'. Km-1, N'. Pm )
                插入( N'. Pm, K' ) 并通过将其他指针和值右移使之成为
                    N 中的第一个指针和值
                用 N'. Km-1 替换 parent( N ) 中的 K'
            end
            else begin
                令 m 满足: ( N'. Pm, N'. Km ) 是 N' 中的最后一个指针/值对
                从 N' 中去除( N'. Pm, N'. Km )
                插入( N'. Pm, N'. Km ) 并通过将其他指针和值右移使之成为
                    N 中的第一个指针和值
                用 N'. Km 替换 parent( N ) 中的 K'
            end
        end
        else ...与 then 的情况对称...
    end
end
end
end procedure
```

图 11-19 从 B⁺ 树中删除索引项

算法可参见教材

如何利用B+树进行检索?

如何利用B+树进行增加和删除操作?

B+树的增加和删除操作时如何进行分裂和合并?

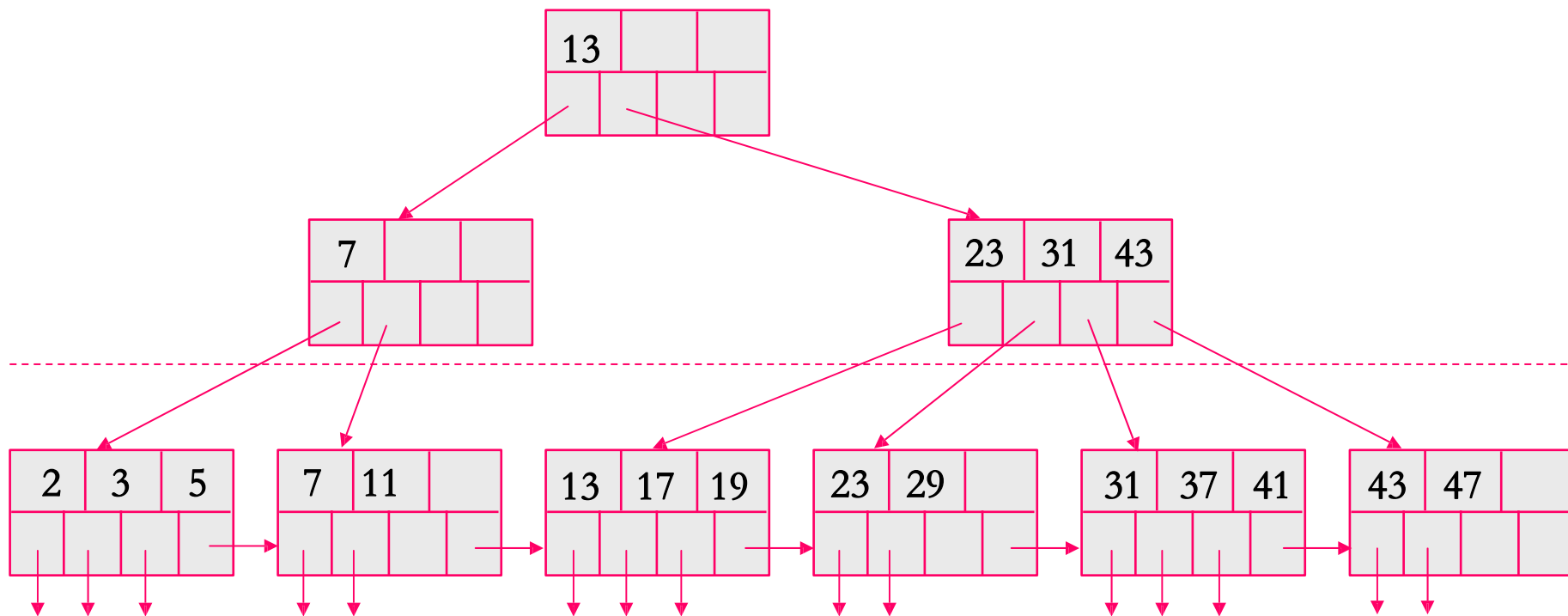
什么条件下分裂与合并?

分裂与合并时如何调整B+树的指针?



B+树之键值插入与结点分裂过程示意

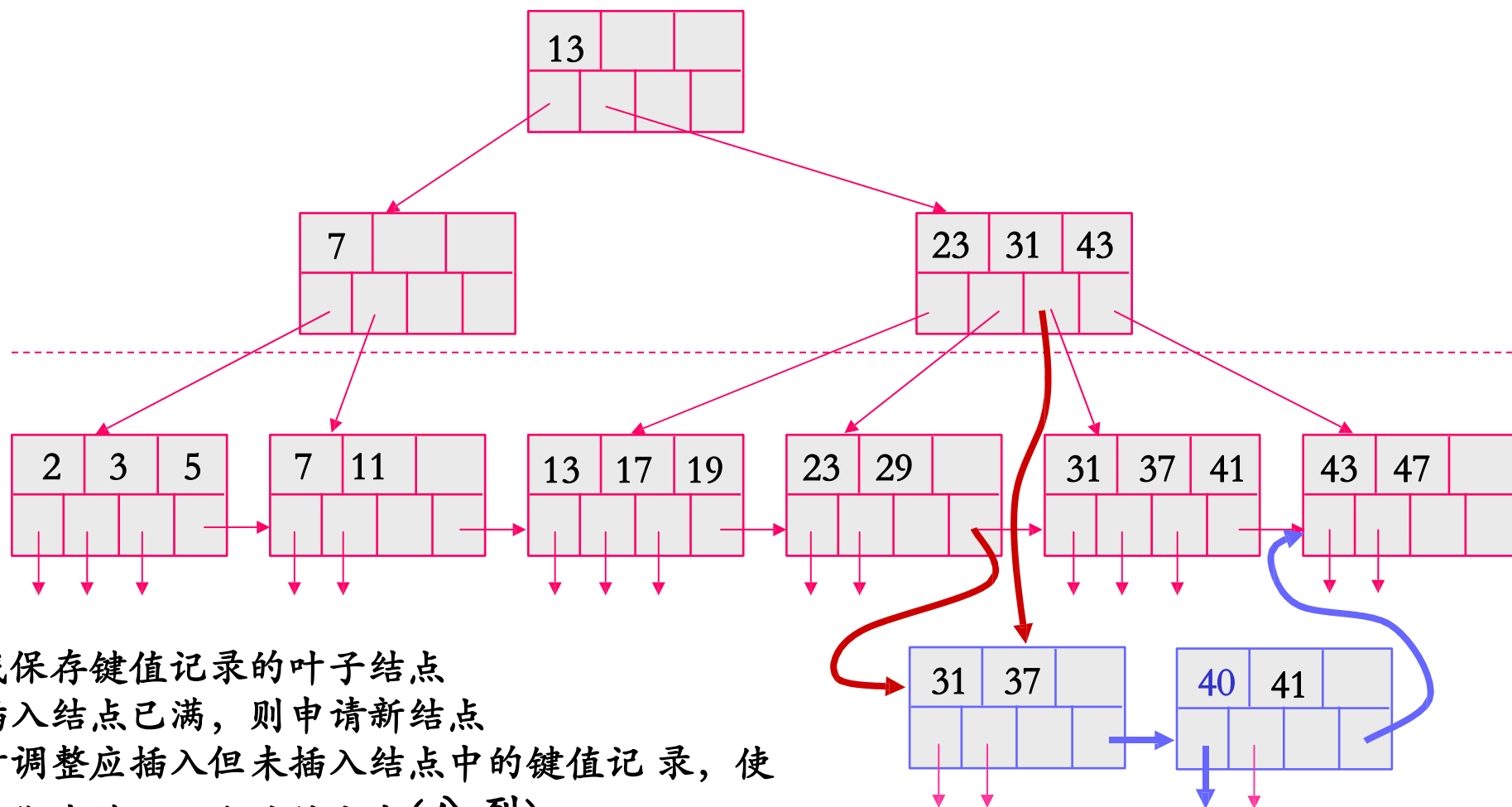
问题1：在下列B+树中插入键值为40的记录





B+树之键值插入与结点分裂过程示意

插入键值为40的记录: 进行中

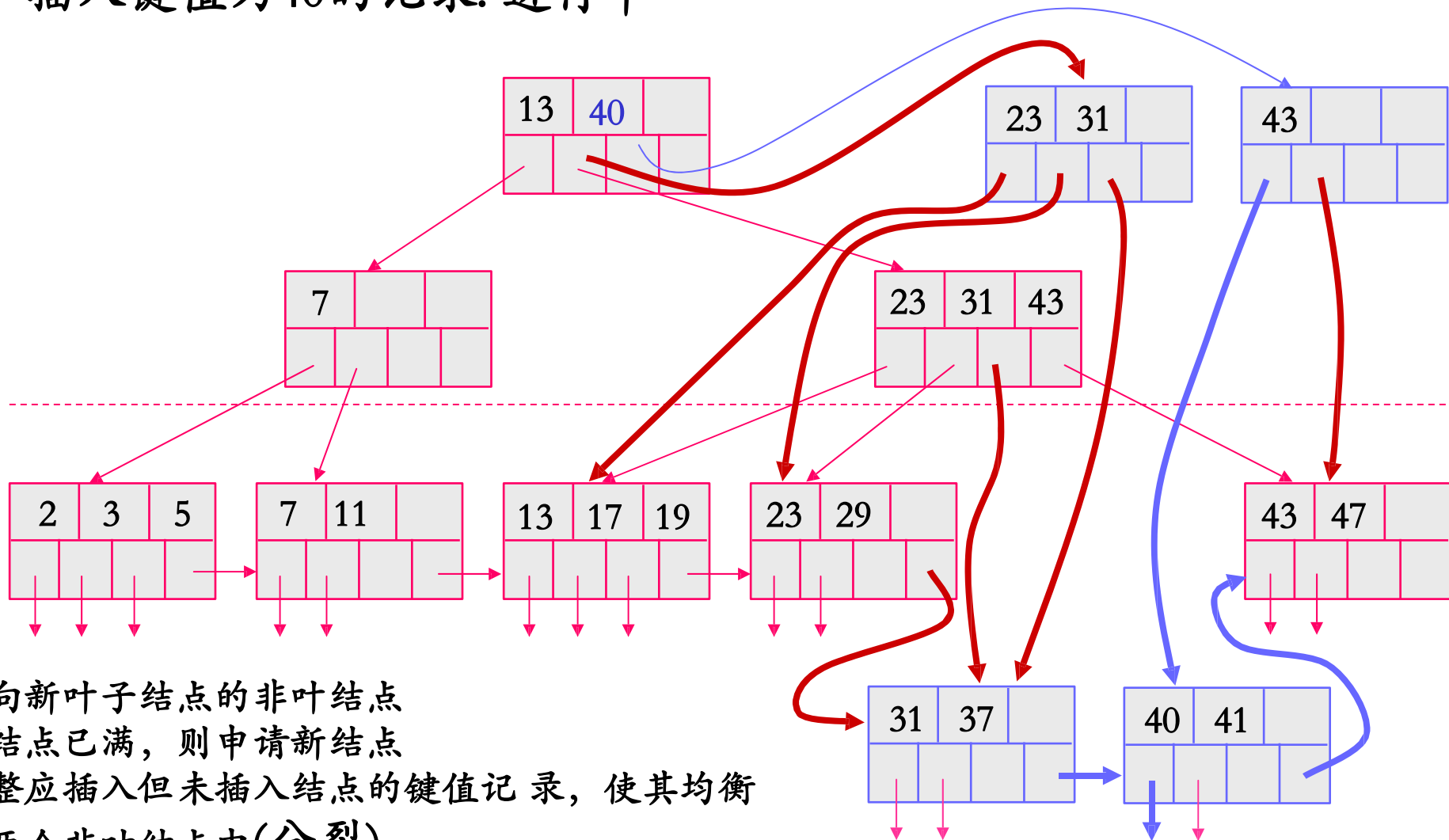


1. 寻找保存键值记录的叶子结点
2. 应插入结点已满，则申请新结点
3. 同时调整应插入但未插入结点中的键值记录，使其均衡存放于两个叶结点中(分裂)
4. 调整指针使其指向新叶子结点



B+树之键值插入与结点分裂过程示意

插入键值为40的记录: 进行中

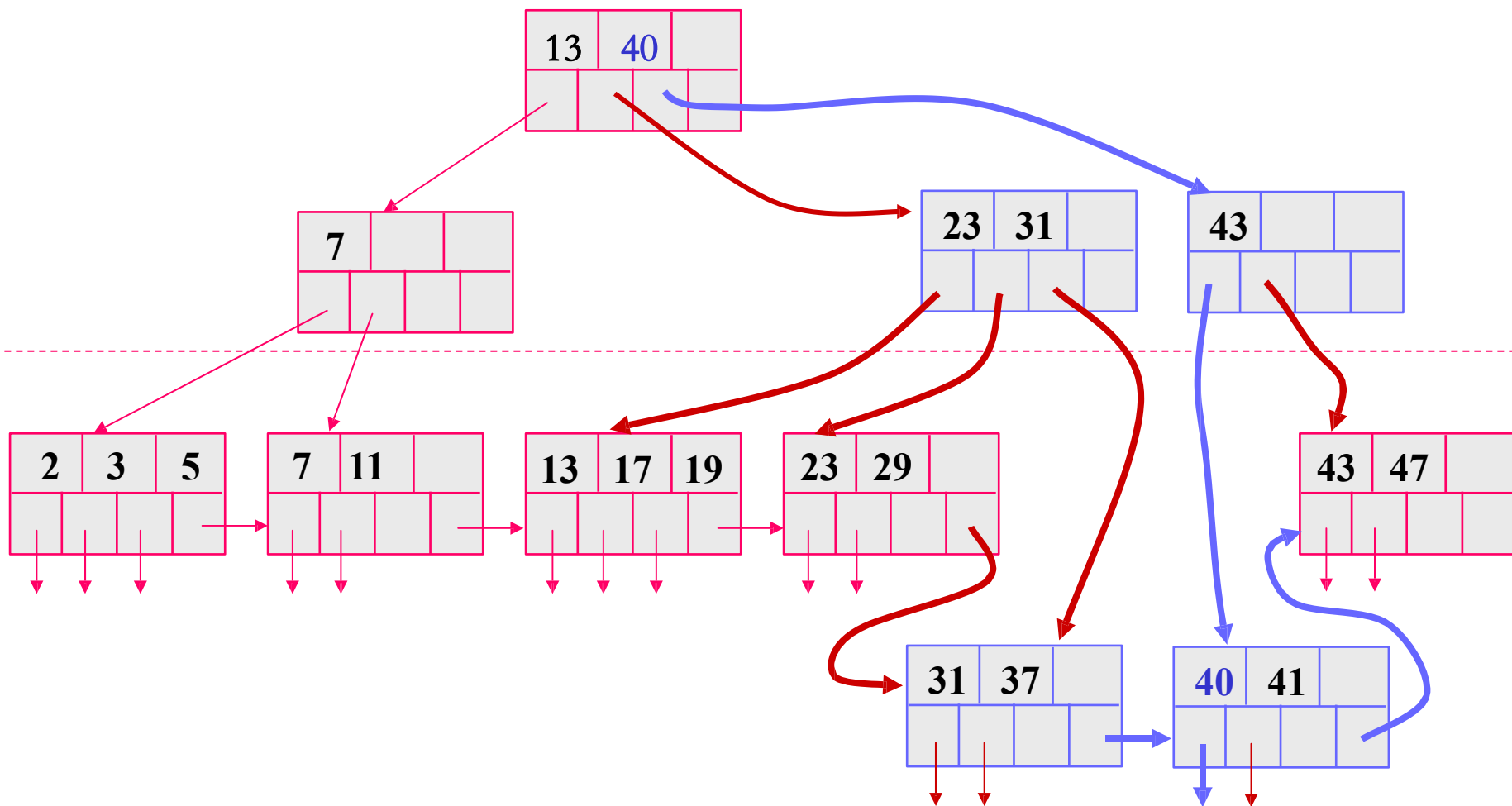


5. 寻找指向新叶子结点的非叶结点
6. 应插入结点已满，则申请新结点
7. 同时调整应插入但未插入结点的键值记录，使其均衡存放于两个非叶结点中(分裂)
8. 调整各结点指针使其指向正确



B+树之键值插入与结点分裂过程示意

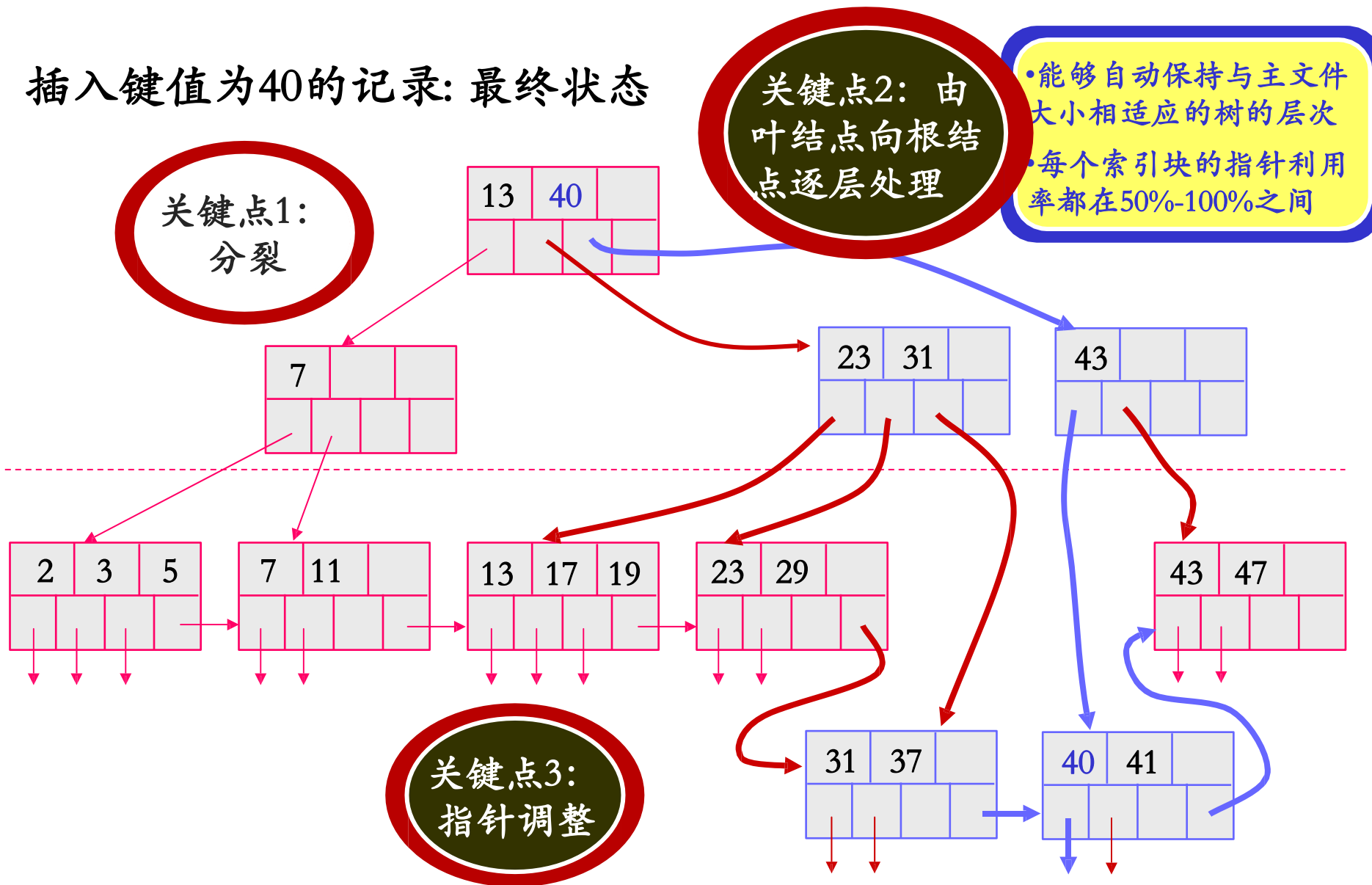
插入键值为40的记录: 最终状态





B+树之键值插入与结点分裂过程示意

插入键值为40的记录: 最终状态





B+树之键值插入与结点分裂过程示意

将算法表达出来

```
procedure insert(value K, pointer P)
  if (树为空) 创建一个空叶结点 L, 同时它也是根结点
  else 找到应该包含值 K 的叶结点 L
  if (L 所含搜索码少于 n-1 个)
    then insert_in_leaf(L, K, P)
  else begin /* L 已经含有 n-1 个搜索码了, 分裂 L */
    创建结点 L'
    把 L.P1, ..., L.Kn-1 复制到可以存储 n 个(指针, 搜索码)对的内存块 T 中
    insert_in_leaf(T, K, P)
    令 L'.Pn = L.Pn; 令 L.Pn = L'
    从 L 中删除 L.P1, ..., L.Pn-1, L.Kn-1, L.Pn
    把 T.P1, ..., T.K⌈n/2⌋} 从 T 复制到 L 中, L 以 L.P1 作为开始
    把 T.K⌈n/2⌋} + 1, ..., T.Kn 从 T 复制到 L' 中, L' 以 L'.P1 作为开始
    令 K' 表示 L' 中最小搜索码值
    insert_in_parent(L, K', L')
  end

procedure insert_in_leaf(node L, value K, pointer P)
  if (K 比 L.K1 小)
    then 把 P、K 插入 L 中, 紧接在 L.P1 前面
  else begin
    令 Ki 表示 L 中小于等于 K 的最大搜索码值
    把 P、K 插入 L 中, 紧跟在 L.Ki 前面
  end

procedure insert_in_parent(node N, value K', pointer N')
  if (N 是树的根结点)
    then 在 L 中插入(V, P)
  else begin
    创建结点 R 包含 N、K'、N' /* N 和 N' 都是指针 */
    令 R 成为树的根结点
    return
  end

  令 P = N 的父结点
  if (P 包含的指针少于 n 个)
    then 将 (K', N') 插到 N 后面
  else begin /* 分裂 P */
    将 P 复制到可以存放 P 以及 (K', N') 的内存块 T 中
    创建结点 (K', N') 插入 T 中, 紧跟在 N 后面
    删除所有 P 中所有项; 创建结点 P'
    把 T.P1, ..., T.K⌈n/2⌋} 复制到 P 中
    令 K'' = T.K⌈n/2⌋} + 1
    把 T.P⌈n/2⌋} + 1, ..., T.Pn 复制到 P' 中
    insert_in_parent(P, K'', P')
  end
end
```

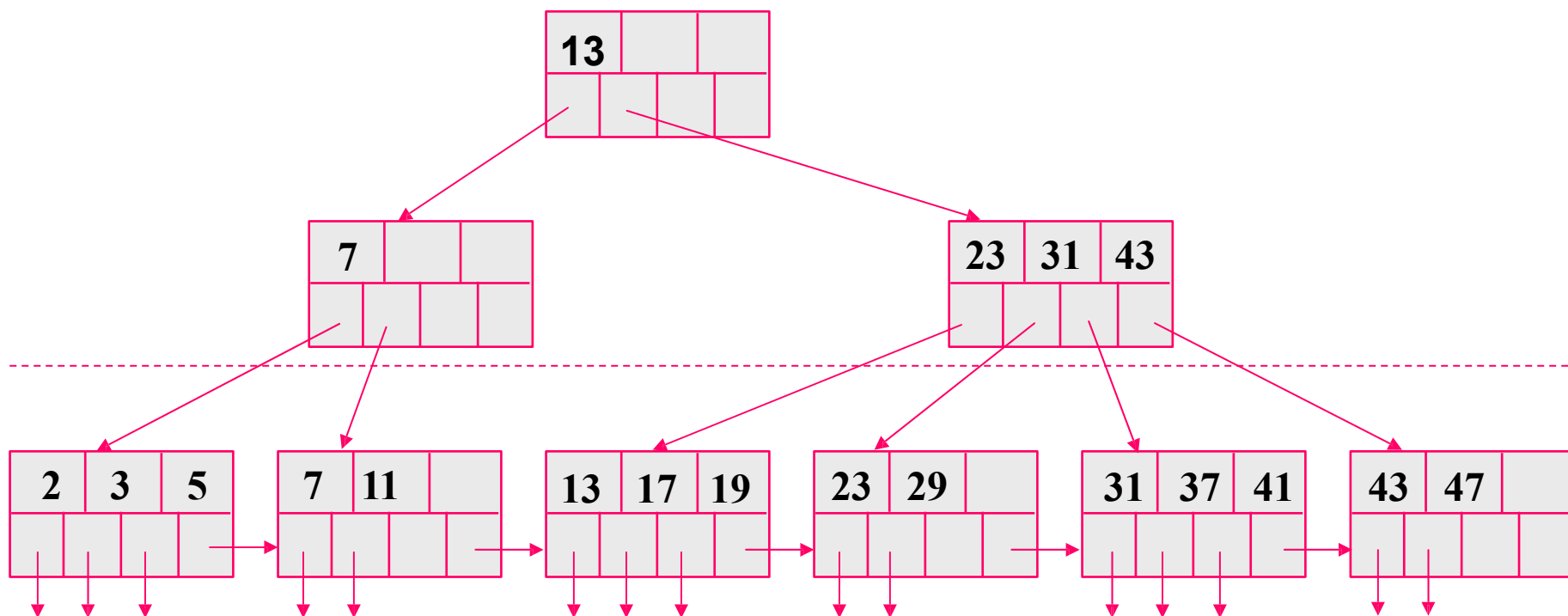
此算法可
参见教材

图 11-15 往 B* 树中插入索引项



B+树之键值删除与结点合并过程示意 (1)问题

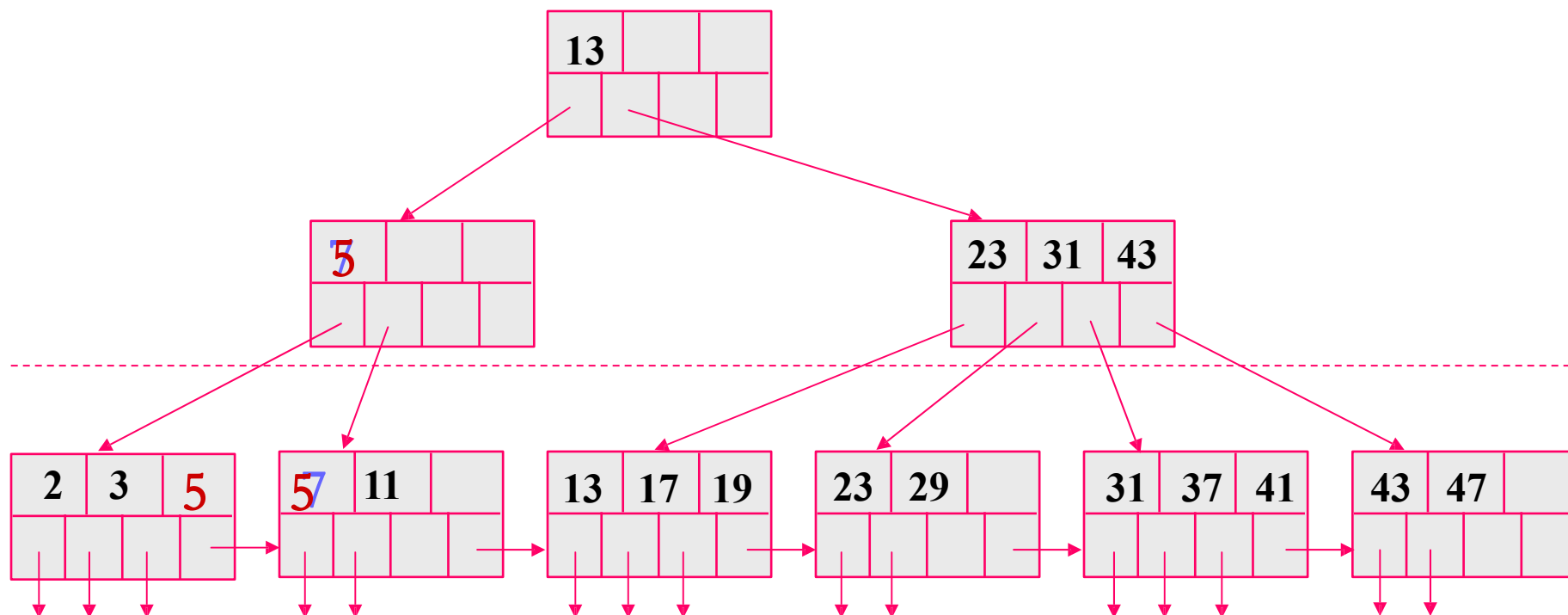
问题1： 在下列B+树中删除键值为7的记录





B+树之键值删除与结点合并过程示意 (2)删除过程与合并过程示意

删除键值为7的记录: 进行中

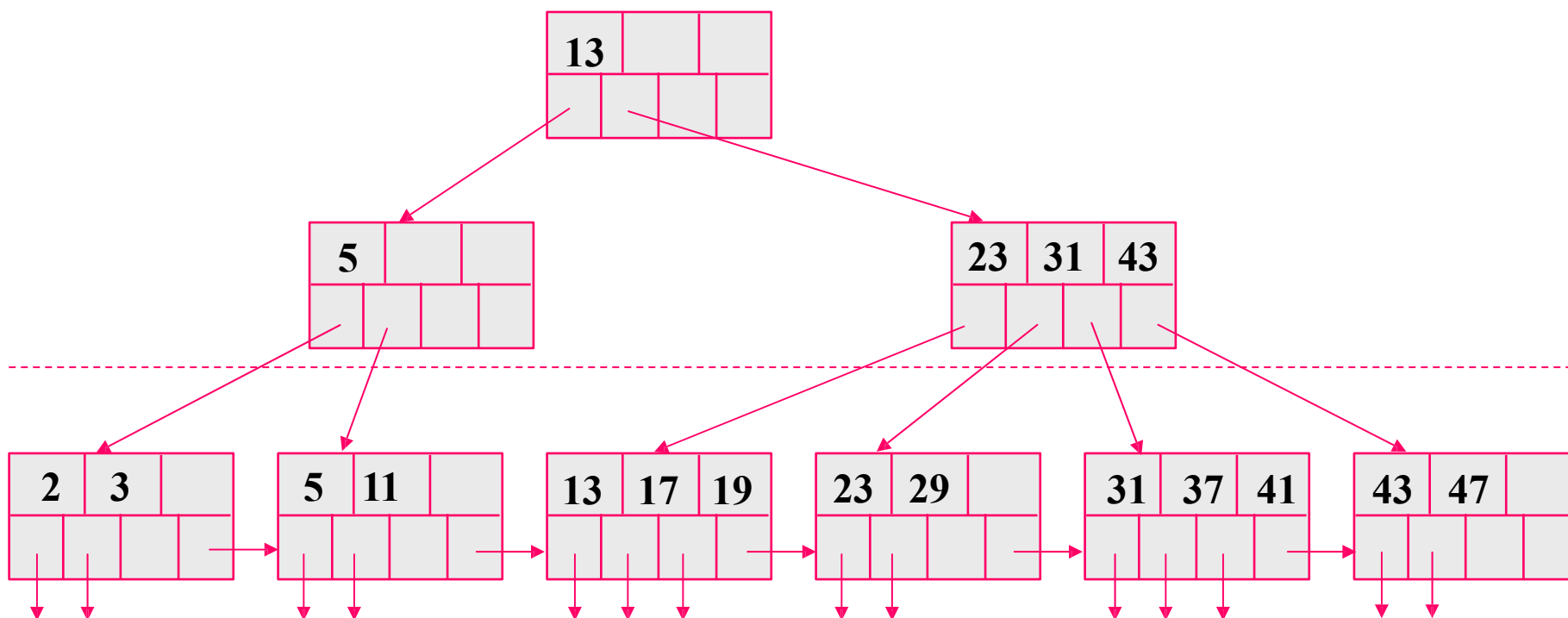


- (1) 叶子结点中寻找等于键值的记录, 删除相应的指针及主文件中对应的记录
- (2) 调整其左侧(或右侧)结点及本结点中的键值记录, 使其均衡存放于两个叶结点中
- (3) 如有调整, 则进一步调整其上层非叶结点, 重新确定其键值, 以满足大于等于 键值的记录都在其右侧



B+树之键值删除与结点合并过程示意 (2)删除过程与合并过程示意

删除键值为7的记录: 最终状态

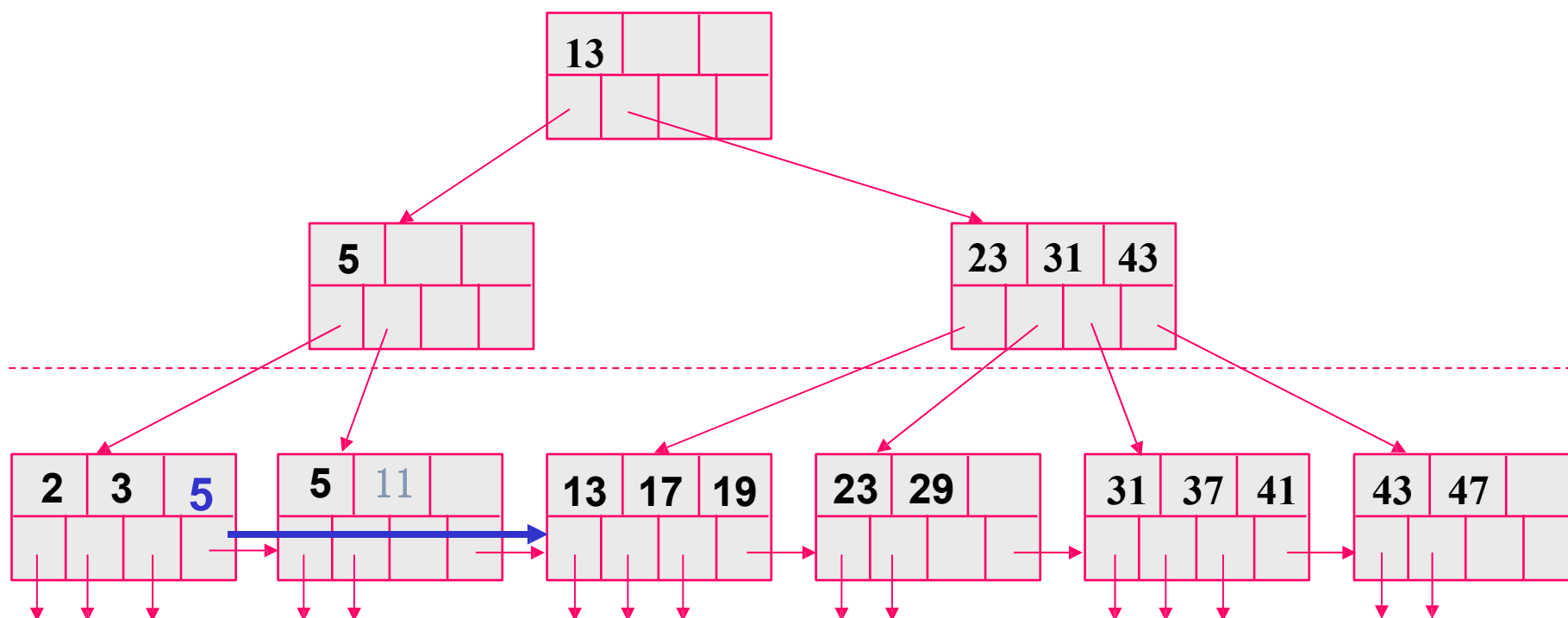


问题2: 在此B+树中删除键值为11的记录



B+树之键值删除与结点合并过程示意 (2)删除过程与合并过程示意

删除键值为11的记录: 进行中

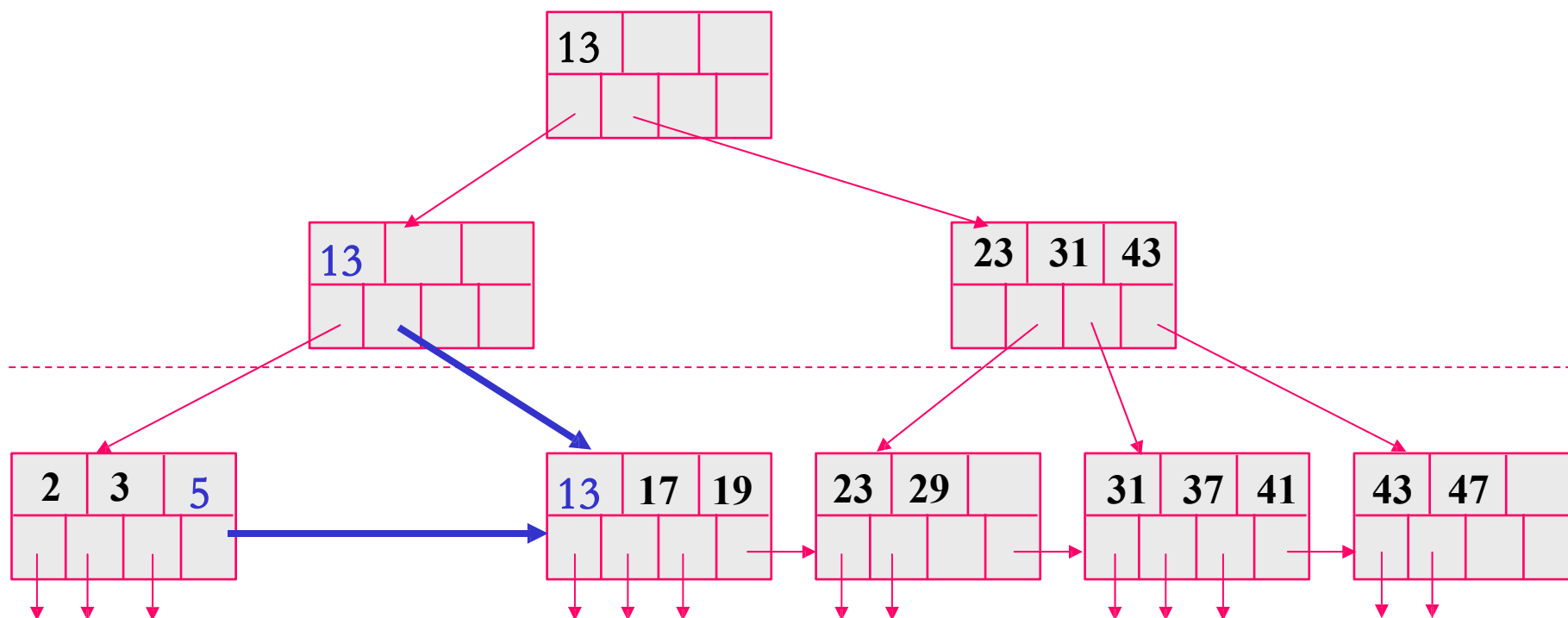


- (1) 叶子结点中寻找等于键值的记录, 删除相应的指针及主文件中对应的记录
- (2) 将本结点的剩余键值, 移动到左侧(或右侧)结点。此空结点将被删除。调整叶结点指针



B+树之键值删除与结点合并过程示意 (2)删除过程与合并过程示意

删除键值为11的记录: 进行中

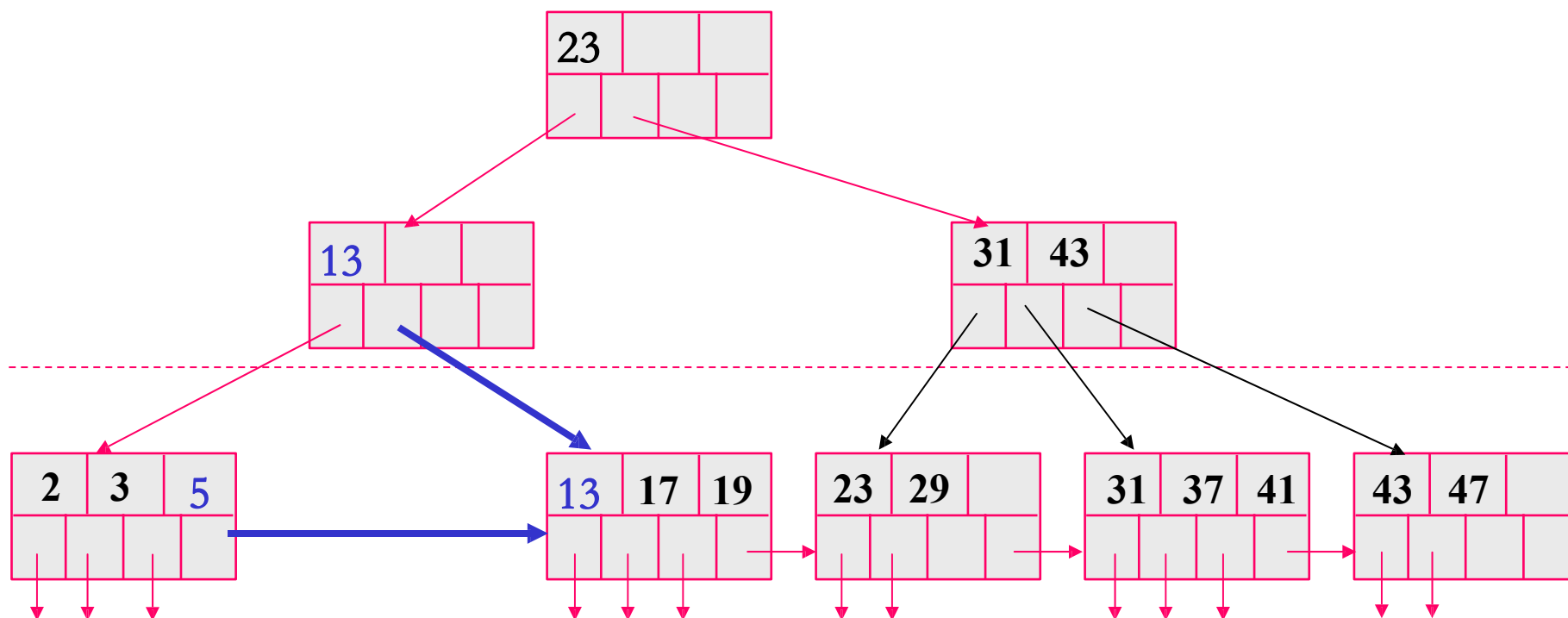


(3) 如有调整, 则进一步调整其上层非叶结点, 重新确定其键值, 以满足大于等于键值的记录都在其右侧。依次向上调整, 直到根结点。



B+树之键值删除与结点合并过程示意 (2)删除过程与合并过程示意

删除键值为11的记录: 进行中

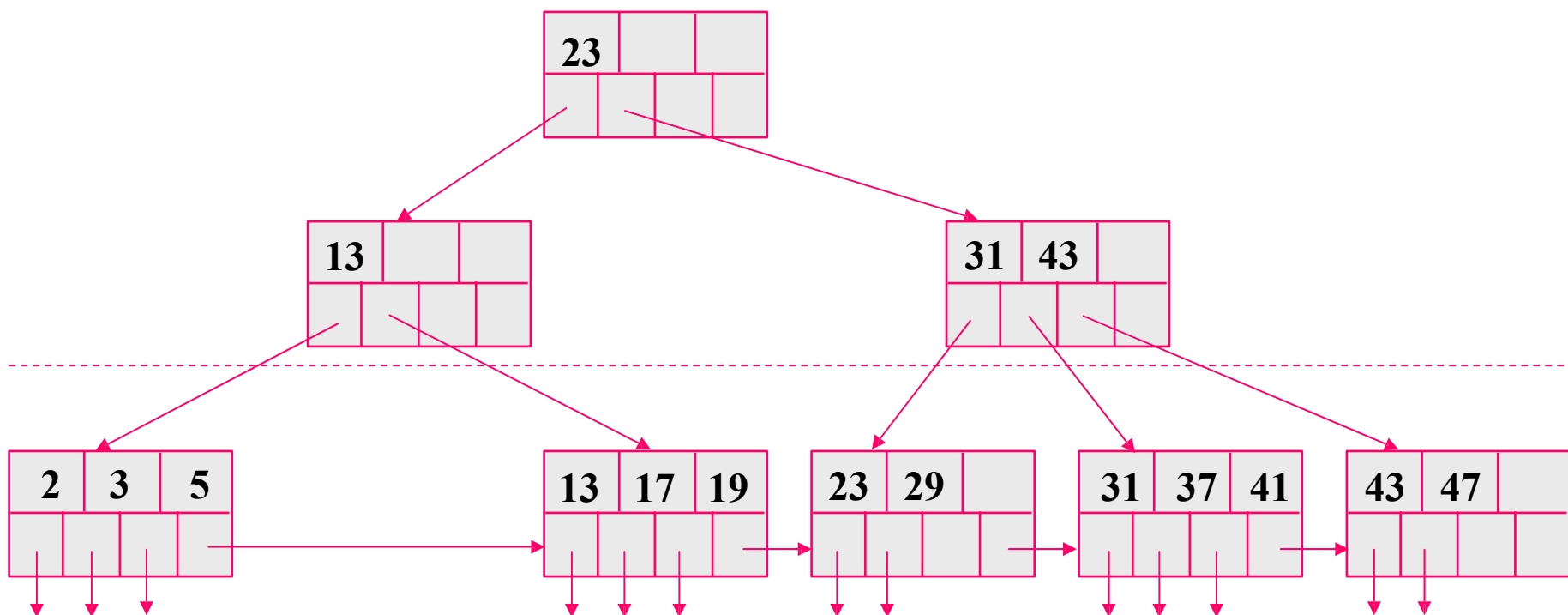


(3) 如有调整, 则进一步调整其上层非叶结点, 重新确定其键值, 以满足大于等于键值的记录都在其右侧。依次向上调整, 直到根结点。



B+树之键值删除与结点合并过程示意 (3)删除过程与合并过程完成之后

删除键值为11的记录: 最终状态

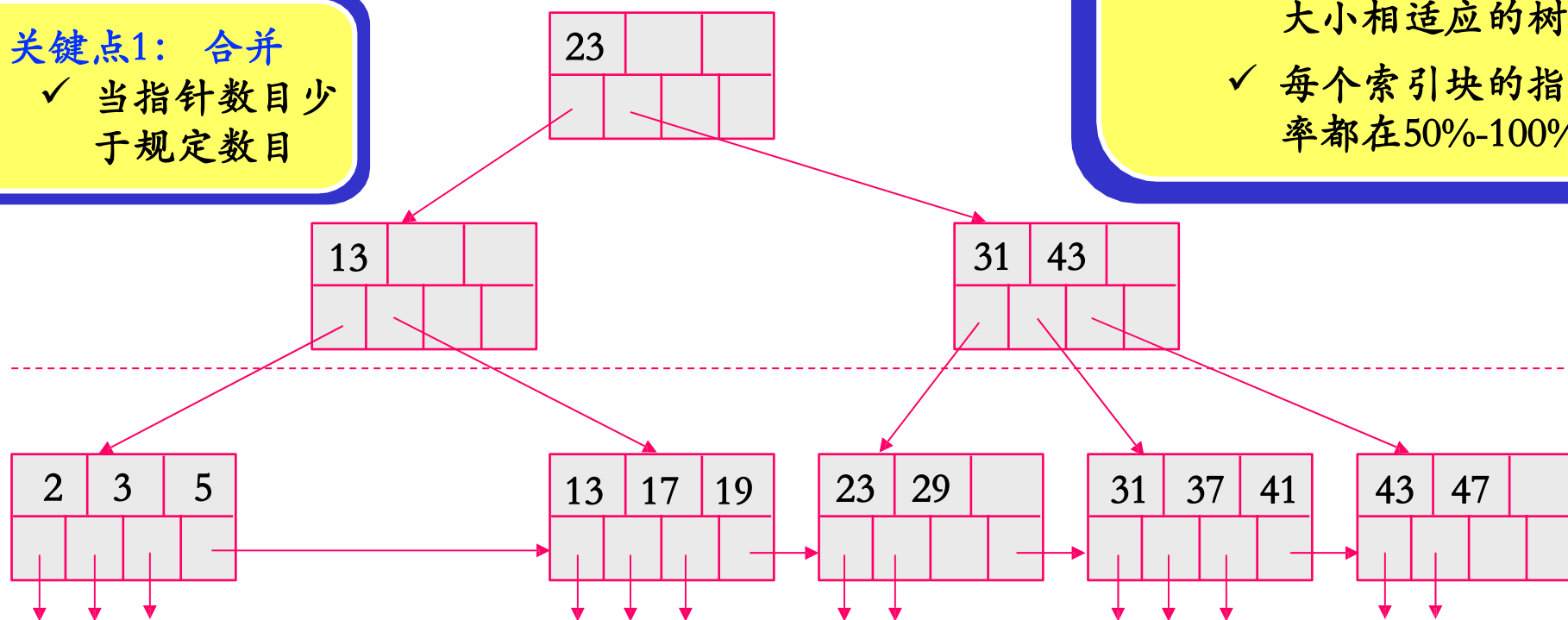




B+树之键值删除与结点合并过程示意 (3)删除过程与合并过程完成之后

删除键值为11的记录: 最终状态

- 关键点1: 合并
 - ✓ 当指针数目少于规定数目



- 关键点3: 指针调整

- 关键点2: 由叶结点向根结点逐层处理
 - ✓ 能够自动保持与主文件大小相适应的树的层次
 - ✓ 每个索引块的指针利用率都在50%-100%之间



B+树之键值删除与结点合并过程示意 4)相关的算法表达

将算法表达出来

此算法可
参见教材
图 11-19

procedure delete (value K , pointer P)

找到包含(K , P)的叶结点 L

delete_entry(L , K , P)

procedure delete_entry(node N , value K , pointer P)

从 N 中删除(K , P)

if (N 是根结点 **and** N 只剩下一个子结点)

then 使 N 的子结点成为该树的新的根结点并删除 N

else if (N 中值/指针太少) **then begin**

令 N' 为 $parent(N)$ 的前一子女或后一子女

令 K' 为 $parent(N)$ 中指针 N 和 N' 之间的值

if (N 和 N' 中的项能放在一个结点中)

then begin /* 合并结点 */

if (N 是 N' 的前一个结点) **then** swap_variables(N , N')

if (N 不是叶结点)

then 将 K' 以及 N 中所有指针和值附加到 N' 中

else 将 N 中所有(K_i , P_i) 对附加到 N' 中; 令 $N'.P_n = N.P_n$

delete_entry($parent(N)$, K' , N); 删除结点 N

end

else begin /* 重新分布: 从 N' 借一个索引项 */

if (N' 是 N 的前一个结点) **then begin**

if (N 是非叶结点) **then begin**

令 m 满足: $N'.P_m$ 是 N' 的最后一个指针

从 N' 中去除($N'.K_{m-1}$, $N'.P_m$)

插入($N'.P_m$, K') 并通过将其他指针和值右移使之成为

N 中的第一个指针和值

用 $N'.K_{m-1}$ 替换 $parent(N)$ 中的 K'

end

else begin

令 m 满足: ($N'.P_m$, $N'.K_m$) 是 N' 中的最后一个指针/值对

从 N' 中去除($N'.P_m$, $N'.K_m$)

插入($N'.P_m$, $N'.K_m$) 并通过将其他指针和值右移使之成为

N 中的第一个指针和值

用 $N'.K_m$ 替换 $parent(N)$ 中的 K'

end

end

else ...与 **then** 的情况对称...

end

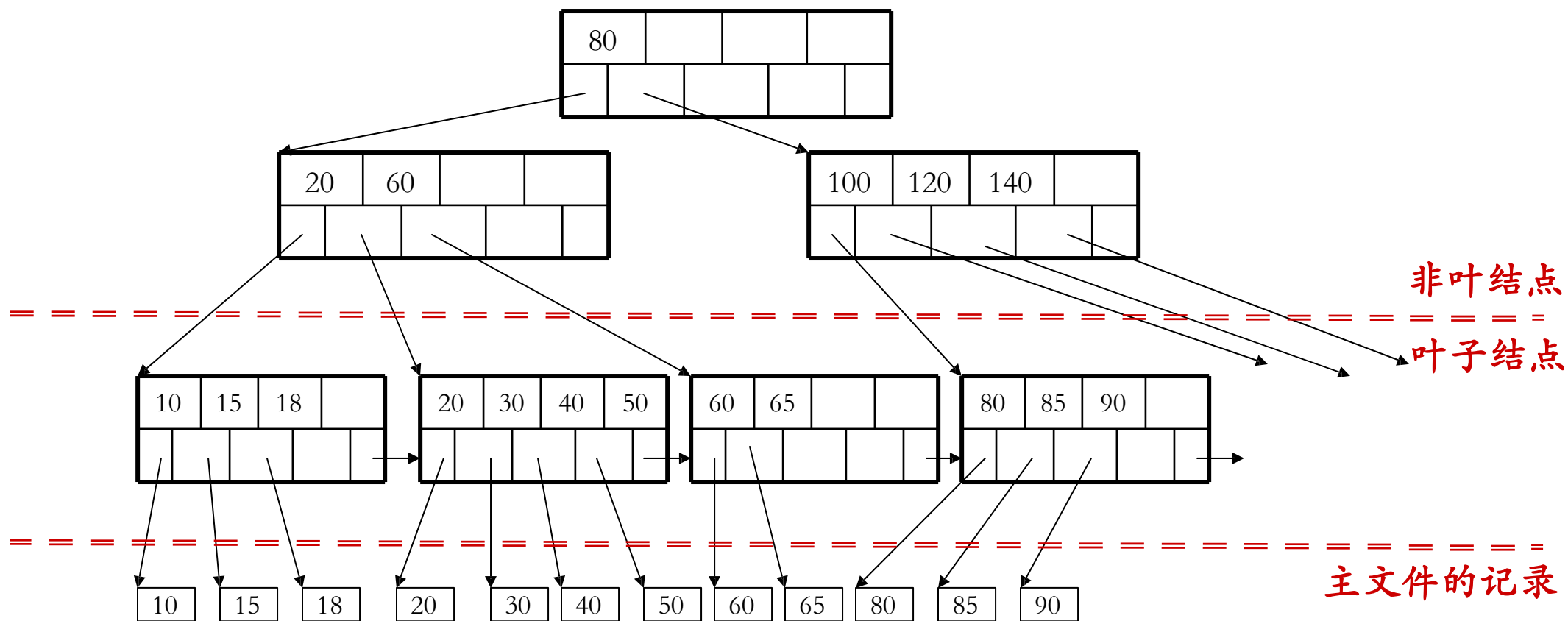
end

end procedure



B+树之分裂与合并过程再示例 (1)背景

示例：初始B+树

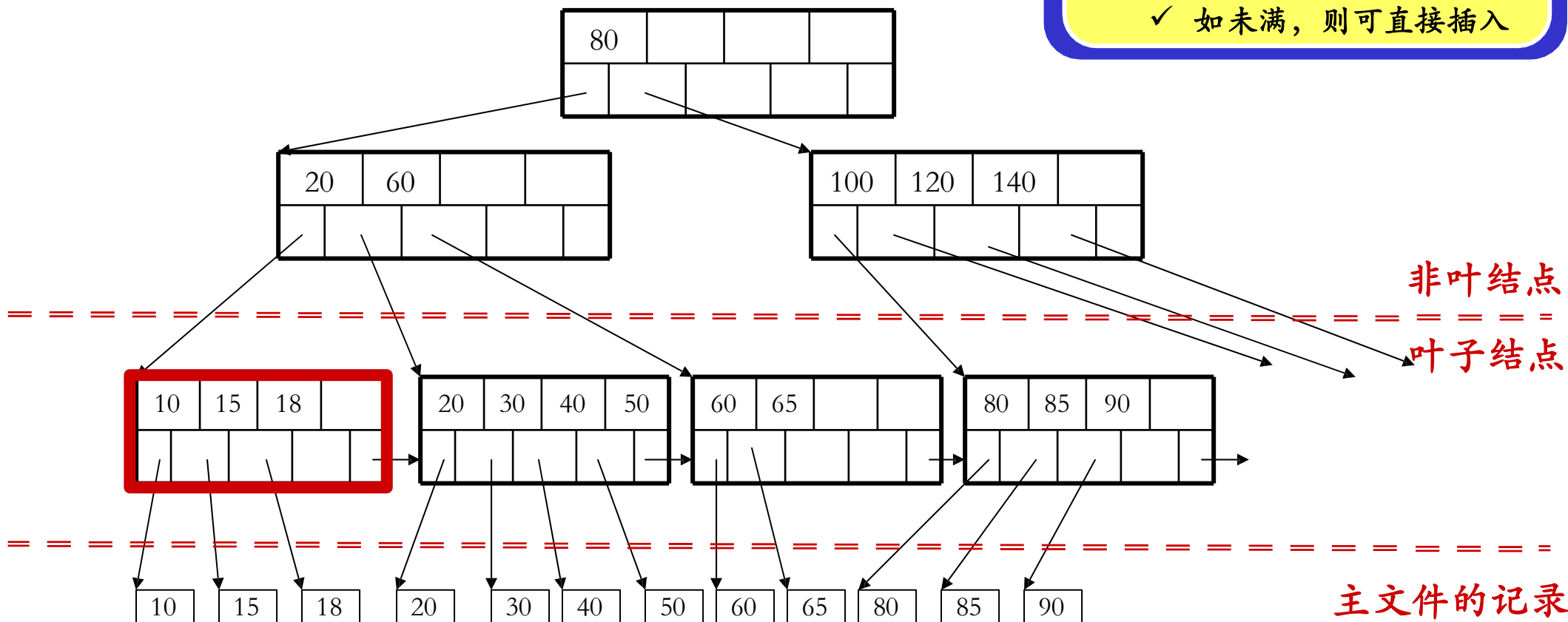




B+树之分裂与合并过程再示例 (2)问题

问题：插入键值为19的记录

- 先定位待插入键值的叶子结点
从根结点开始向下；
- 检查叶子结点是否已满？
✓ 如未滿，则可直接插入



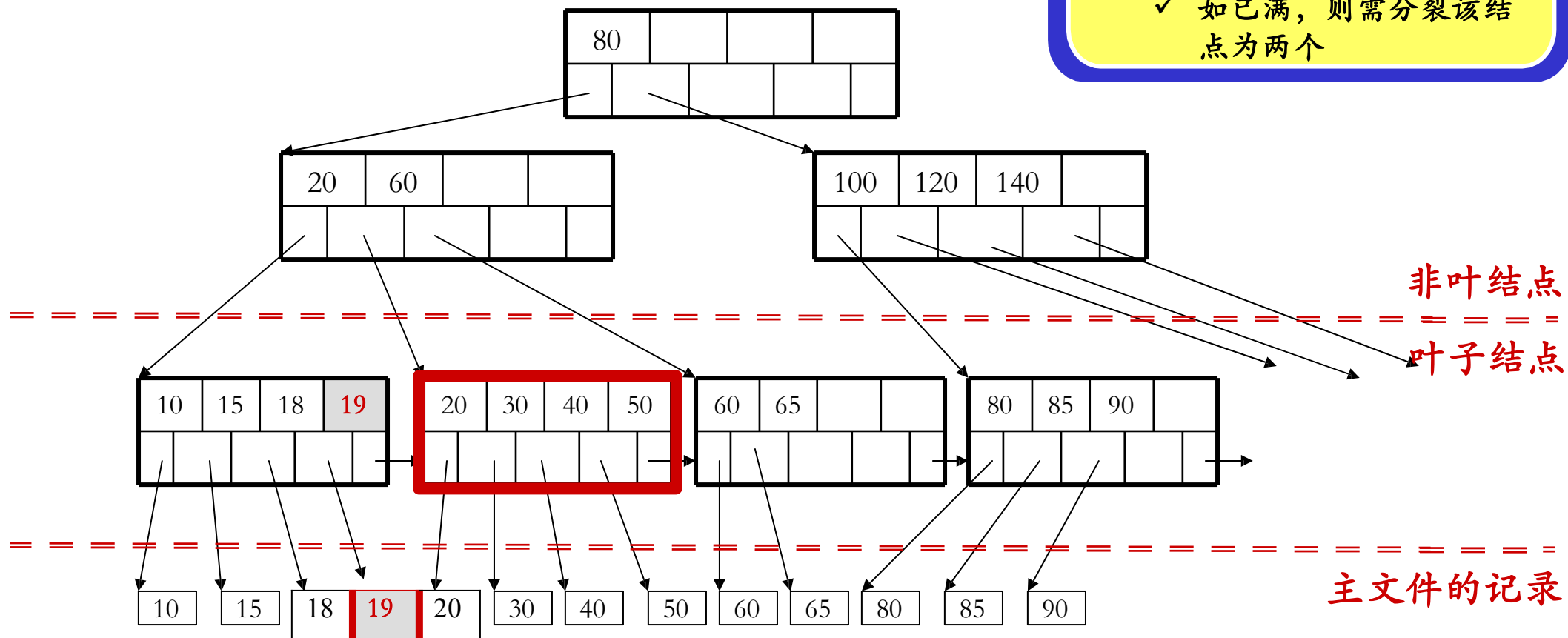


B+树之分裂与合并过程再示例 (3) 复杂插入

键值为19的记录插入后

问题：再插入键值为25的记录

- 先定位待插入键值的叶子结点
从根结点开始向下；
- 检查叶子结点是否已满？
 - ✓ 如已满，则需分裂该结点为两个

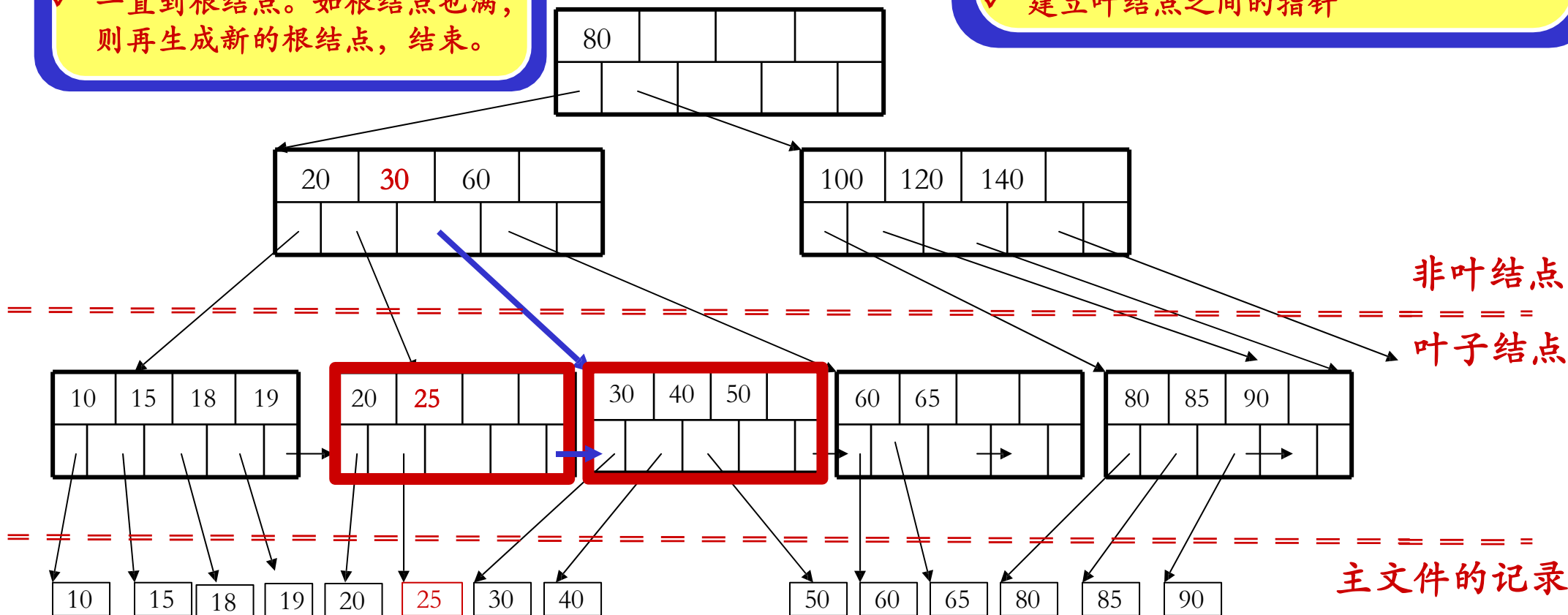




B+树之分裂与合并过程再示例 (4)分裂条件与分裂过程

- ✓ 如父结点已满，则如此继续将其分裂为两个结点，保存相应键值
- ✓ 一直到根结点。如根结点也满，则再生成新的根结点，结束。

- ✓ 分裂结点，将所涉及的键值 准均分为两个结点中存储(包含待插入的)
- ✓ 插入键值为25的键值并指向主文件记录
- ✓ 调整父结点的指针及键值：在父结点中插入键值为30的索引项
- ✓ 建立叶结点之间的指针



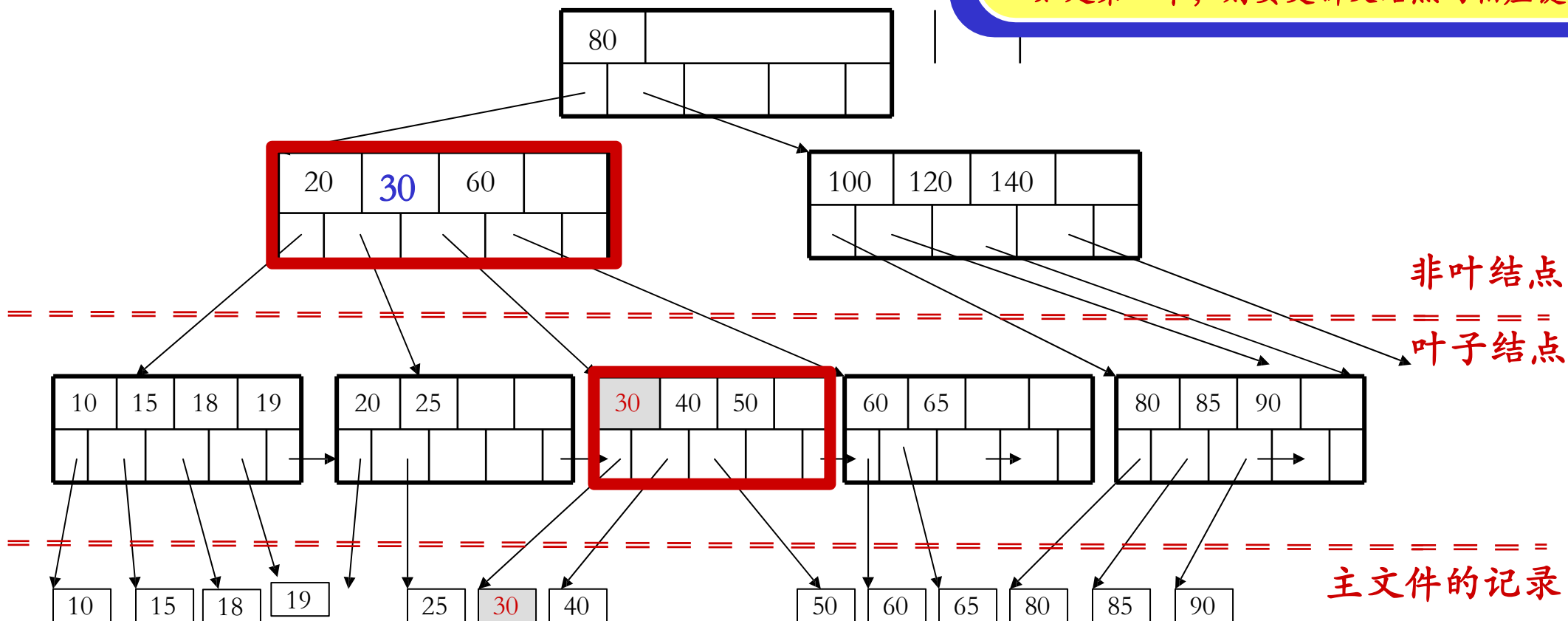


B+树之分裂与合并过程再示例 (5)简单删除

键值为25的记录插入后

问题：删除键值为30的记录

- ✓ 先定位待删除键值的叶子结点：从根结点开始向下；
- ✓ 删除键值及其主文件记录？
- ✓ 如指针数目不少于规定数目，则可以结束
- ✓ 如是第一个，则要更新父结点的相应键值



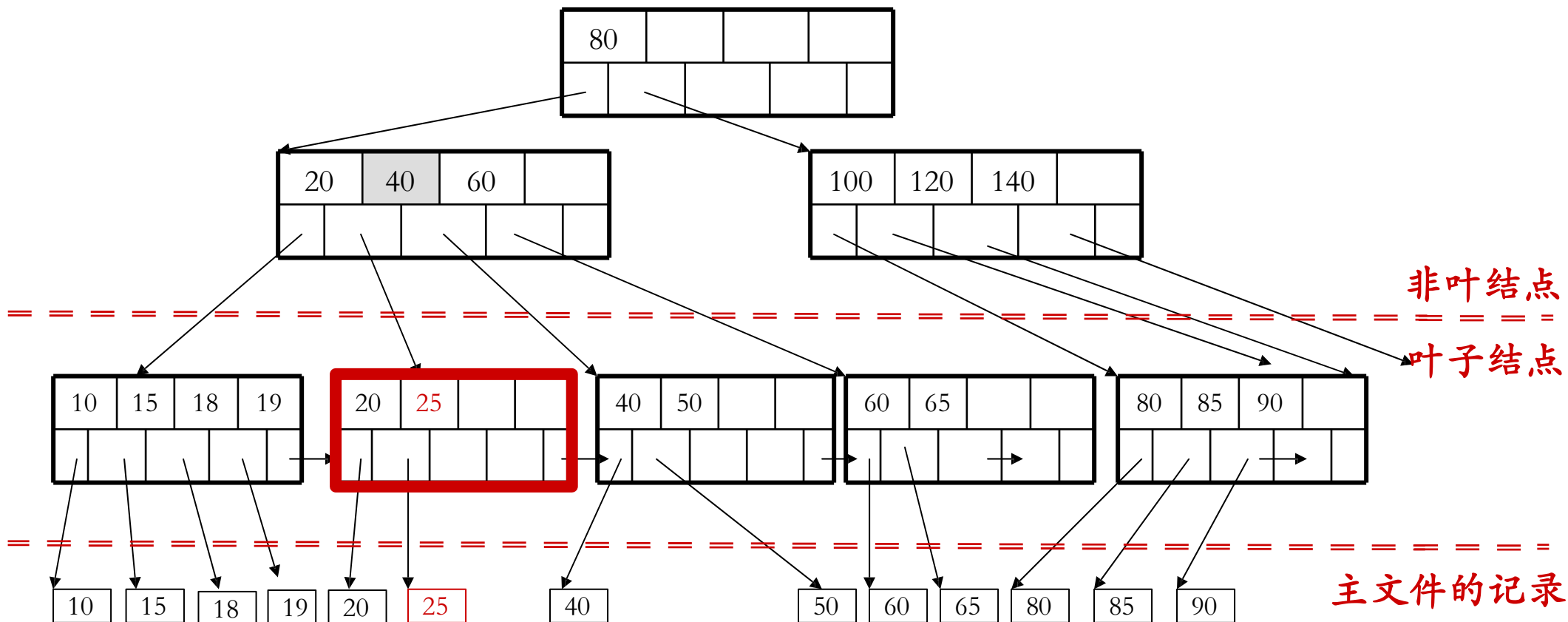


B+树之分裂与合并过程再示例 (6) 复杂删除

键值为30的记录删除后

问题：删除键值为25的记录

✓ 先定位待删除键值的叶子结点：
从根结点开始向下；删除键值及其主文件记录？

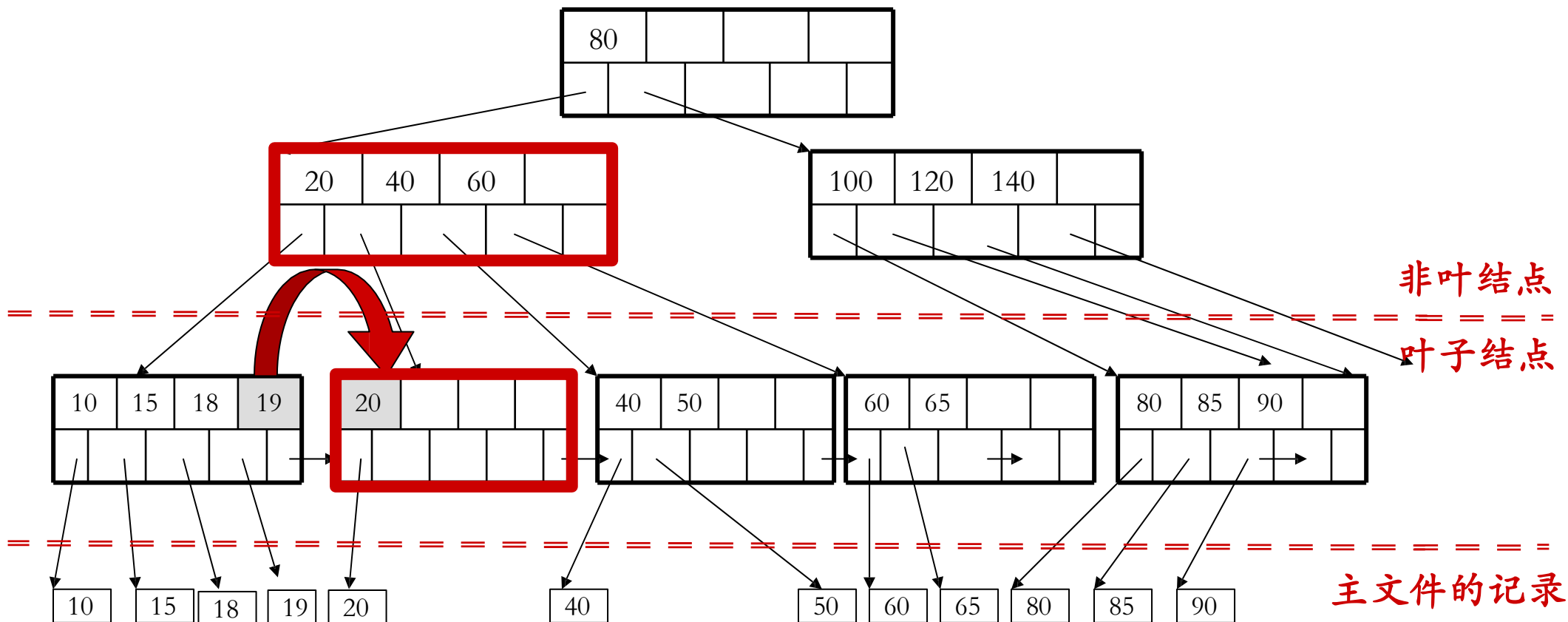




B+树之分裂与合并过程再示例 (7)键值之合并

键值为25的记录删除后

- ✓ 键值删除后，如指针数目少于规定数目，则需要合并
- ✓ 从相邻结点能否转移一些键值到该结点，如可以，则转移；并更新父结点之相应键值



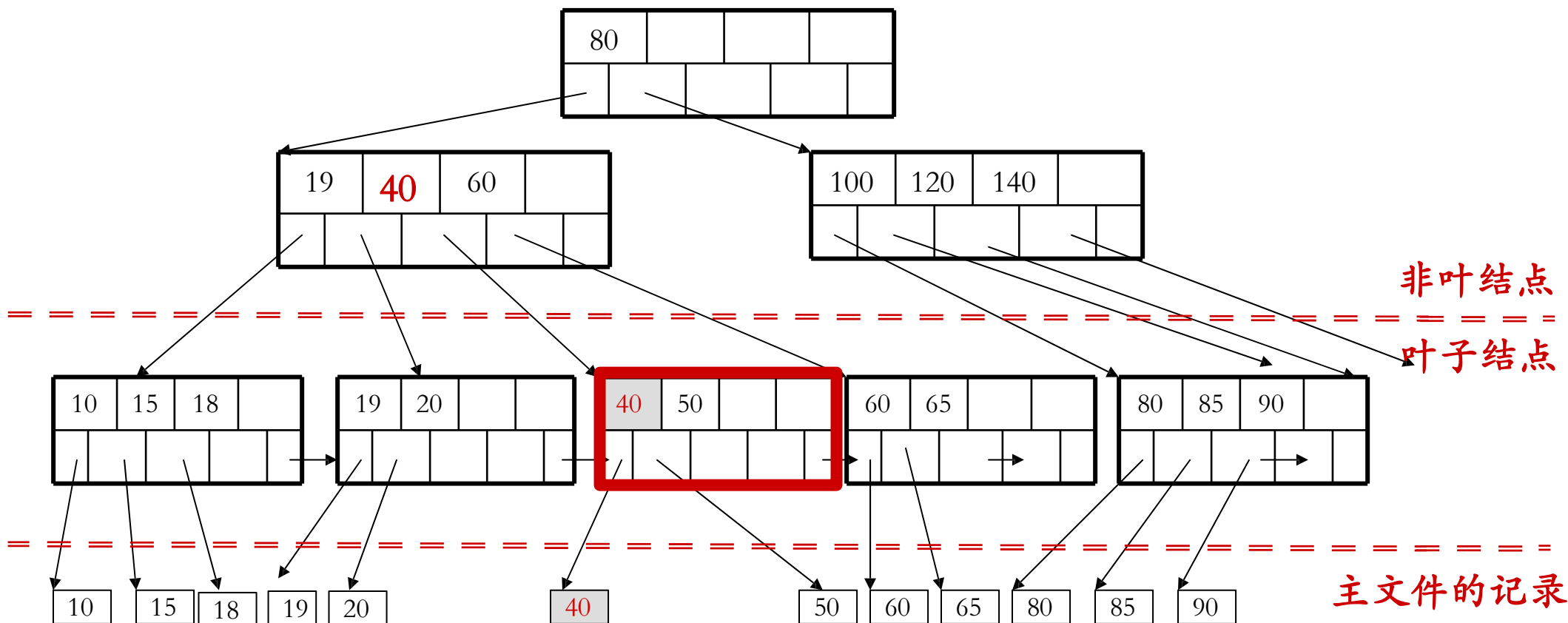


B+树之分裂与合并过程再示例 (8)更复杂之删除

转移键值后

问题：删除键值为40的记录

- ✓ 先定位待删除键值的叶子结点：从根结点开始向下；删除键值及其主文件记录？
- ✓ 如是第一个，则需调整父结点的键值

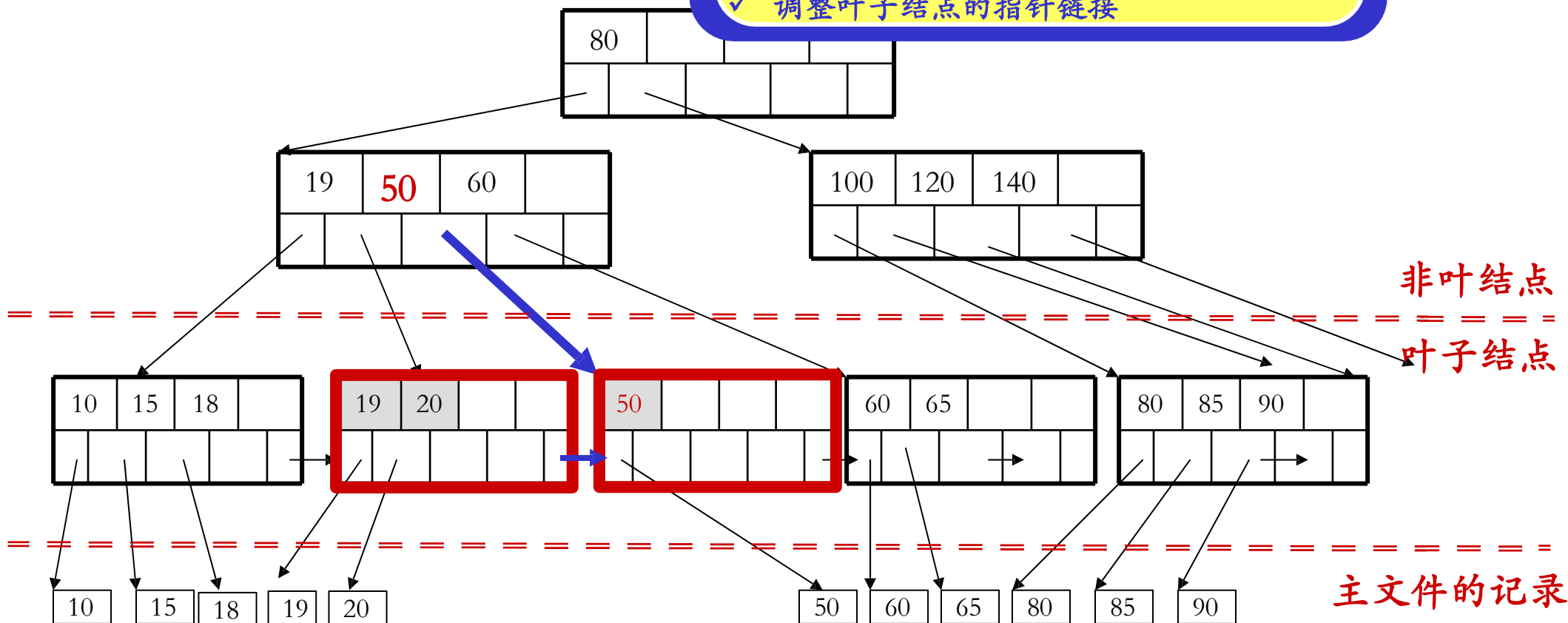




B+树之分裂与合并过程再示例 (9) 结点之合并

键值为40的记录删除后

- ✓ 键值删除后，如指针数目少于规定数目，则需要合并
- ✓ 如不能从相邻结点转移键值到该结点，则考虑结点合并。合并后调整父结点键值及次序。
- ✓ 调整叶子结点的指针链接

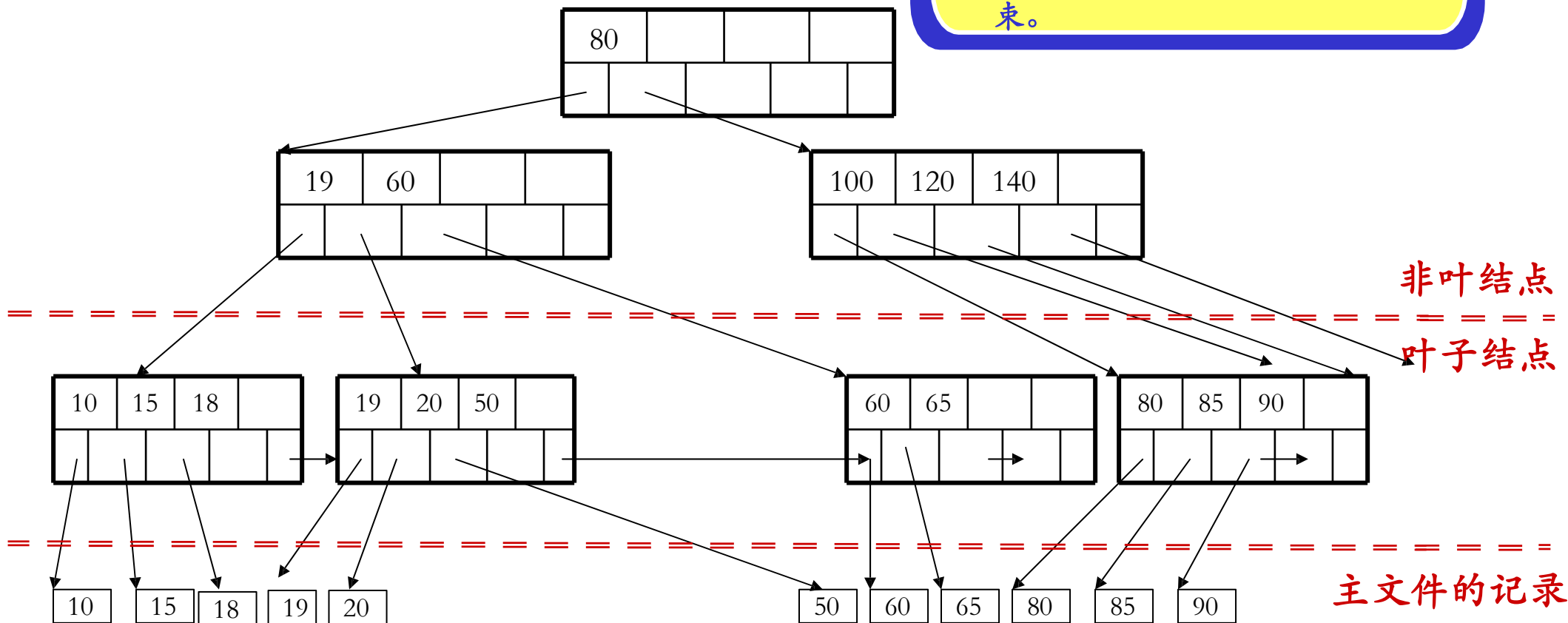




B+树之分裂与合并过程再示例 (10) 合并之连锁反应

结点合并后

- 父结点，如在删除掉索引项及指针后，指针数目小于规定的数目，则继续前面的步骤处理，直至根结点；
- 如指针不小于规定的数目，则结束。





B+树之分裂与合并过程再示例 (11)要点概括

➤ 插入记录时:

- ✓ 可能需要分裂: 当索引块充满时, 需要分裂。
- ✓ 分裂可能引发连锁反应, 由叶结点直至根结点
- ✓ 分裂后需要仔细调整索引键值及指针。注意叶子结点之间链接关系的调整。

➤ 删除记录时:

- ✓ 可能需要合并: 合并记录—从相邻结点中转移索引项(仍旧是两个结点块)
- ✓ 可能需要合并: 两个结点块合并成一个结点块。
- ✓ 合并可能引发连锁反应, 由叶结点直至根结点。
- ✓ 合并后需要仔细调整索引键值及指针。
- ✓ 注意叶子结点之间链接关系的调整。